第七章 图

7.1 抽象数据类型图的定义

7.2 图的存储表示

7.3 图的遍历

7.4 最小生成树

7.5 重（双）连通图和关节点

7.6 两点之间的最短路径问题

7.7 拓扑排序

7.8 关键路径

图的结构定义:

图是由一个顶点集 V 和一个弧集 R构成的数据结构。

Graph = (V , R)

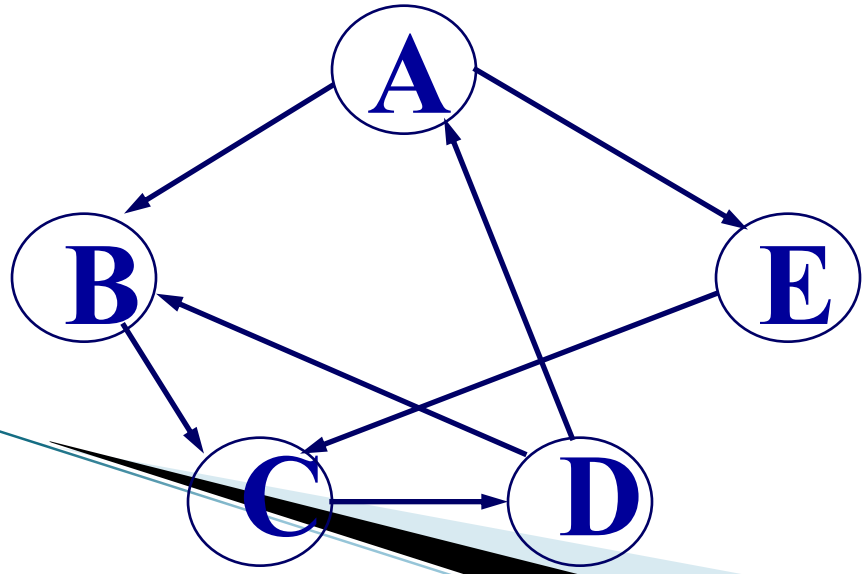
其中，VR = {<v,w>| v,w∈V 且 P(v,w)}

<v,w>表示从 v 到 w 的一条弧，并称 w 为**弧头**， v 为**弧尾**。

谓词 P(v,w) 定义了弧 <v,w>的意义或信息。

由于“弧”是有方向的，因此称由顶点集和弧集构成的图为**有向图**。

例如： $G_1 = (V_1, VR_1)$



其中

$V_1=\{A, B, C, D, E\}$

$VR_1=\{<A,B>, <A,E>,$
$<B,C>, <C,D>, <D,B>,$
$<D,A>, <E,C>\}$

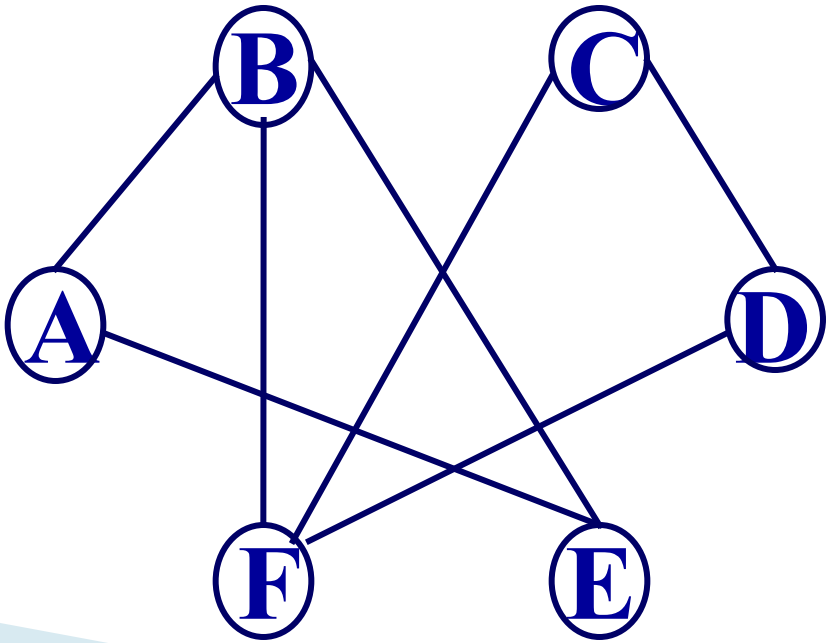
若<v, w>∈VR 必有<w, v>∈VR,则称 (v,w) 为顶点v 和顶点 w 之间存在一条**边**。

由顶点集和边集构成的图称作**无向图**。

例如: $G_2=(V_2,VR_2)$

$V_2=\{A, B, C, D, E, F\}$

$VR_2=\{<A,B>, <A,E>,$
$<B,E>, <C,D>, <D,F>,$
$<B,F>, <C,F>\}$



名词和术语

网、子图 ➡

完全图、稀疏图、稠密图 ➡

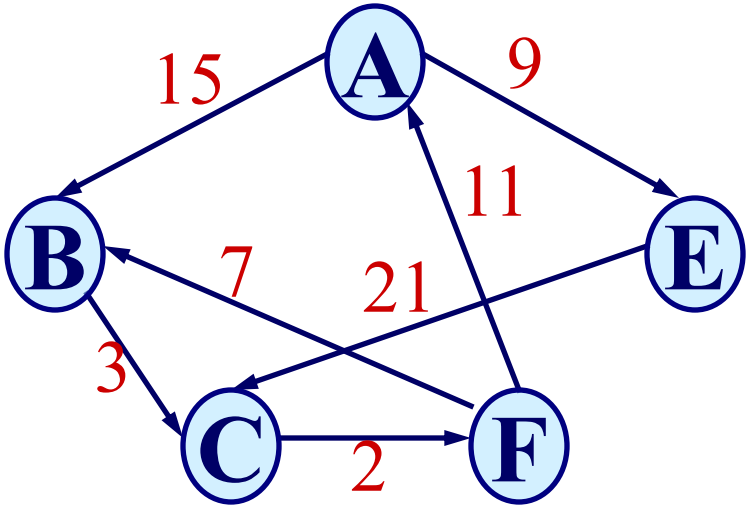
邻接点、度、入度、出度 ➡

路径、路径长度、简单路径、简单回路➡

连通图、连通分量、
强连通图、强连通分量 ➡

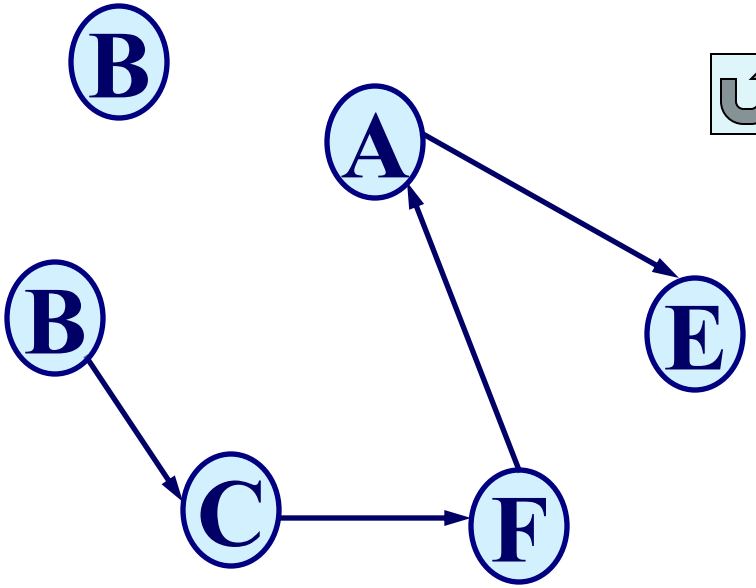
生成树、生成森林 ➡





弧或边带权的图分别称作**有向网**或**无向网**。

设图G=(V,{VR}) 和图 G′=(V′,{VR′}), 且 V′⊆V, VR′⊆VR, 则称 G′ 为 G 的**子图**。



假设图中有 **n** 个顶点，**e** 条边，则

含有 **e=n(n-1)/2** 条边的无向图称作**完全图**；

含有 **e=n(n-1)** 条弧的有向图称作 **有向完全图**；

若边或弧的个数 **e<nlogn**，则称作**稀疏图**，否则称作**稠密图**。

假若顶点v 和顶点w 之间存在一条边，则称顶点v 和w 互为**邻接点**，

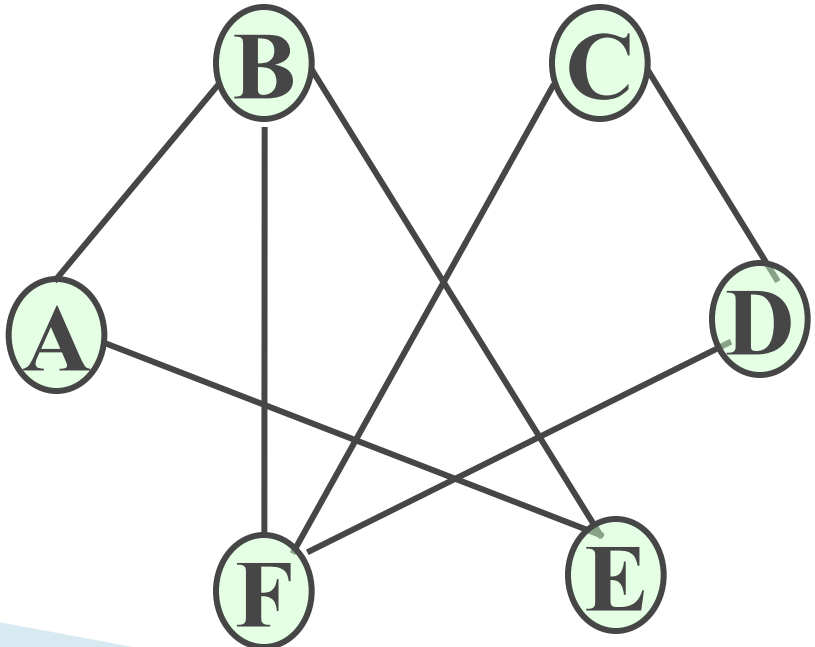
边(v,w) 和顶点v 和w 相**关联**。

和顶点v 关联的**边的数目**定义为边的**度**。

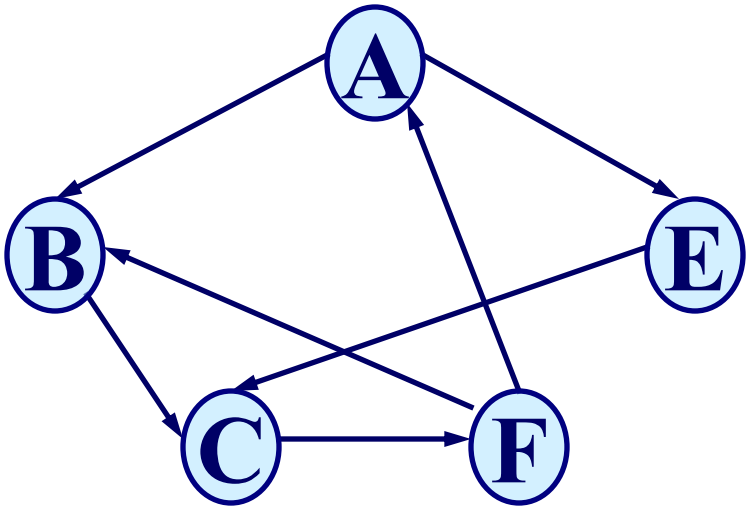
例如:

ID(B) = 3

ID(A) = 2



对有向图来说，



顶点的**出度**: 以顶点v为弧尾的弧的数目；

顶点的**入度**: 以顶点v为弧头的弧的数目。

顶点的**度(TD)=出度(OD)+入度(ID)**

例如:

OD(B) = 1

ID(B) = 2

TD(B) = 3

设图G=(V,{VR})中的一个顶点序列

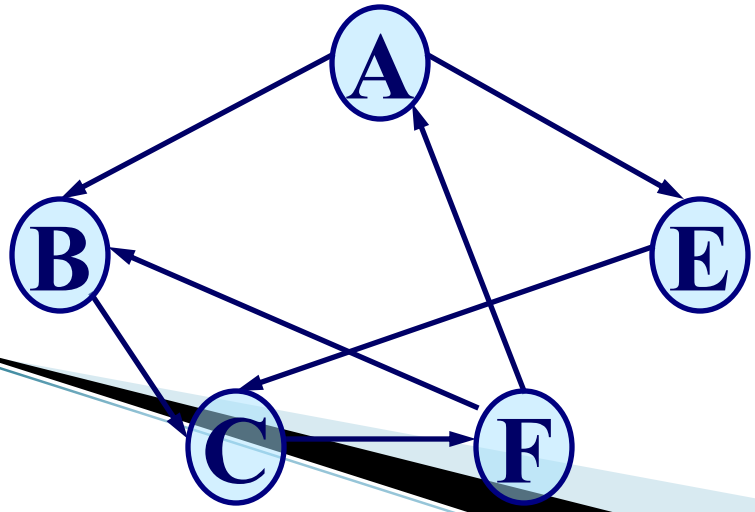
$\{u=v_{i,0},v_{i,1},\ldots,v_{i,m}=w\}$中，$(v_{i,j-1},v_{i,j})\in VR$ $1\le j\le m$,

则称从顶点u 到顶点w 之间存在一条**路径**。

路径上边的数目称作**路径长度**。

如:长度为3的路径

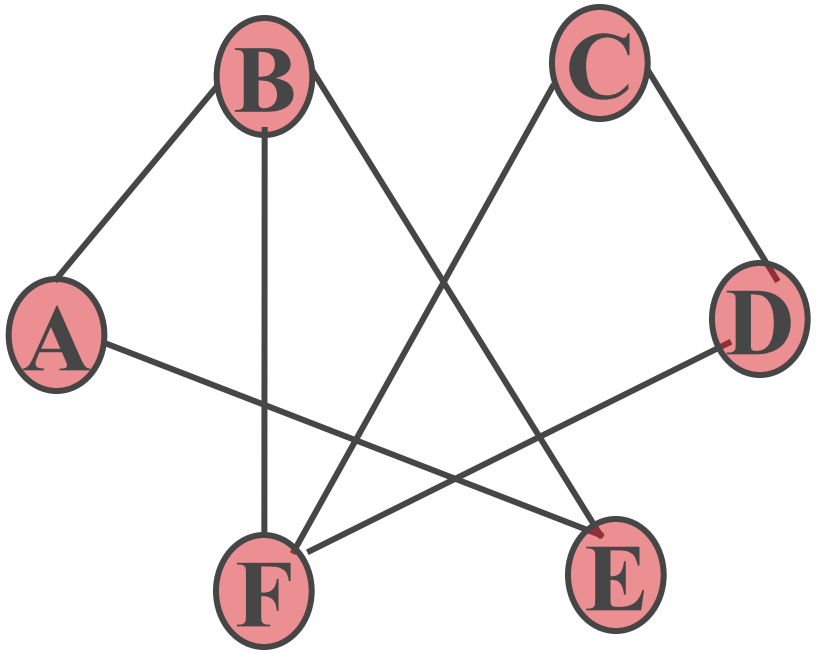
{A,B,C,F}

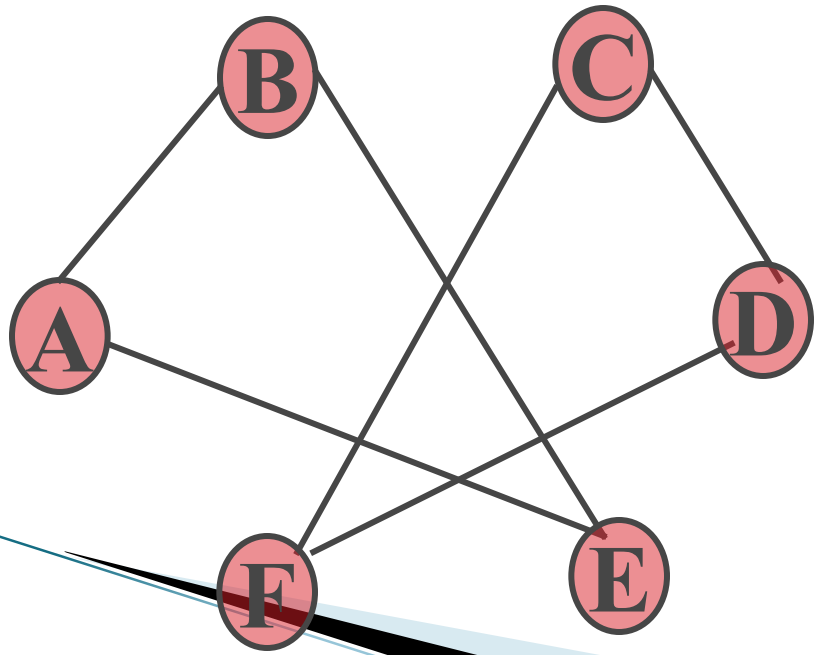


简单路径:序列中顶点不重复出现的路径。

简单回路:序列中第一个顶点和最后一个顶点相同的路径。

若图G中任意两个顶点之间都有路径相通，则称此图为**连通图**；

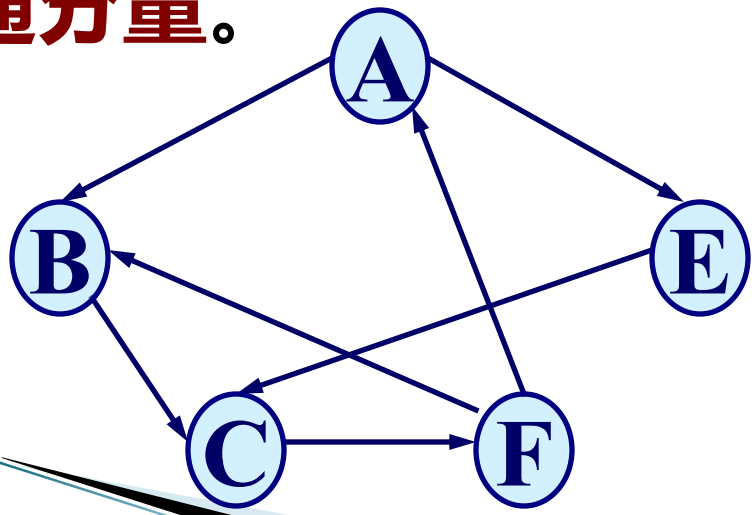


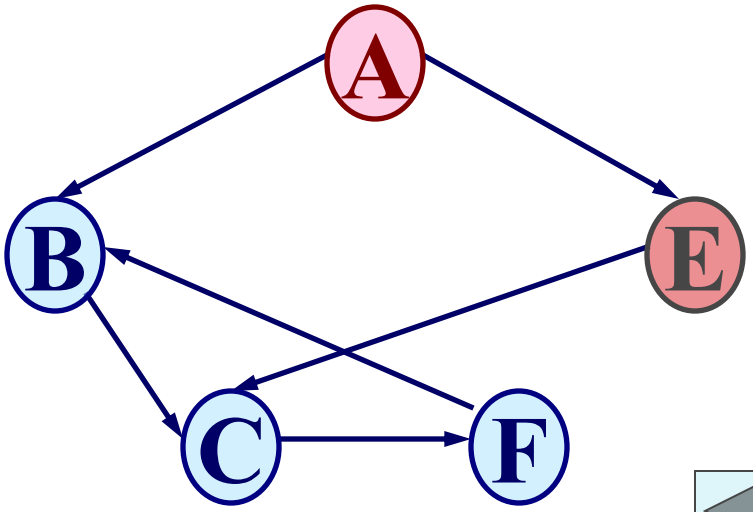


若无向图为非连通图，则图中各个极大连通子图称作此图的**连通分量**。

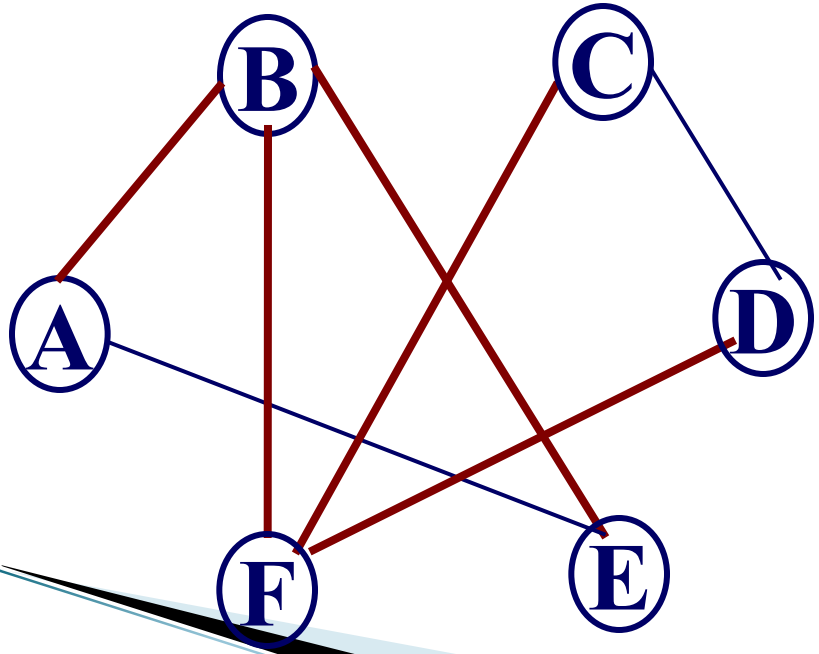
对有向图，若任意两个顶点之间都存在一条有向路径，则称此有向图为**强连通图**。

否则，其各个强连通子图称作它的**强连通分量**。





假设一个连通图有 n 个顶点和 e 条边，其中 n-1 条边和 n 个顶点构成一个极小连通子图，称该极小连通子图为此连通图的**生成树**。



对非连通图，则称由各个连通分量的生成树的集合为此非连通图的**生成森林**。

基本操作

结构的建立和销毁

对顶点的访问操作

插入或删除顶点

插入和删除弧

对邻接点的操作

遍历

结构的建立和销毁

CreatGraph(&G, V, VR):

// 按定义(V, VR) 构造图

DestroyGraph(&G):

// 销毁图



对顶点的访问操作

LocateVex(G, u);

// 若G中存在顶点u，则返回该顶点在

// 图中“**位置**”；否则返回其它信息。

GetVex(G, v); // 返回 v 的值。

PutVex(&G, v, value);

// 对 v 赋值value。

对邻接点的操作

FirstAdjVex(G, v);

// 返回 v 的“**第一个邻接点**”。若该顶点
//在 G 中没有邻接点，则返回“空”。

NextAdjVex(G, v, w);

// 返回 v 的（相对于 w 的）“**下一个邻接**
// **点**”。若 w 是 v 的最后一个邻接点，则
// 返回“空”。



插入或删除顶点

InsertVex(&G, v);

//在图G中增添新顶点v。

DeleteVex(&G, v);

// 删除G中顶点v及其相关的弧。

插入和删除弧

InsertArc(&G, v, w);

// 在G中增添弧<v,w>，若G是无向的，

//则还增添对称弧<w,v>。

DeleteArc(&G, v, w);

//在G中删除弧<v,w>，若G是无向的，

//则还删除对称弧<w,v>。



遍 历

DFSTraverse(G, v, Visit());

//从顶点v起**深度优先**遍历图G，并对每

//个顶点调用函数Visit一次且仅一次。

BFSTraverse(G, v, Visit());

//从顶点v起**广度优先**遍历图G，并对每

//个顶点调用函数Visit一次且仅一次。



7.2 图的存储表示

一、图的数组(邻接矩阵)存储表示

二、图的邻接表存储表示

三、有向图的十字链表存储表示

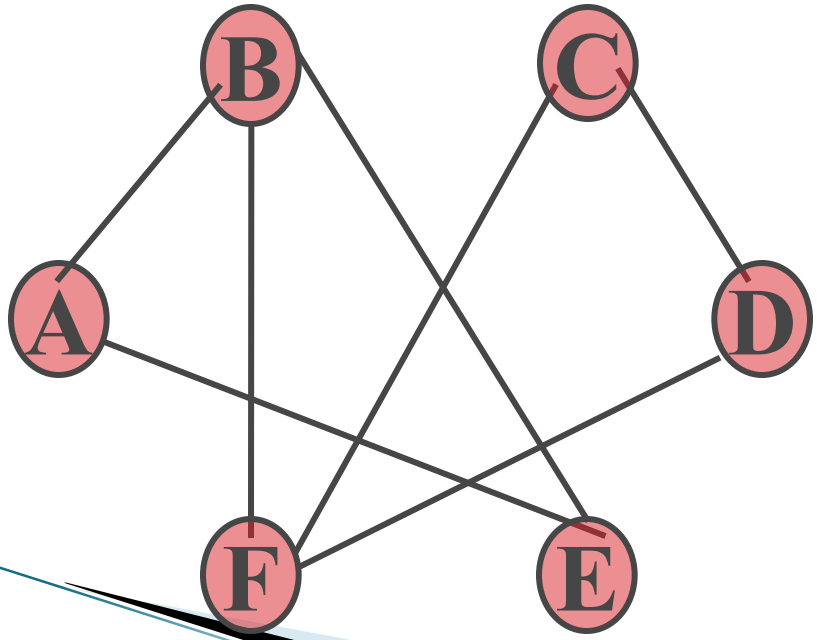
四、无向图的邻接多重表存储表示



一、图的数组(邻接矩阵)存储表示

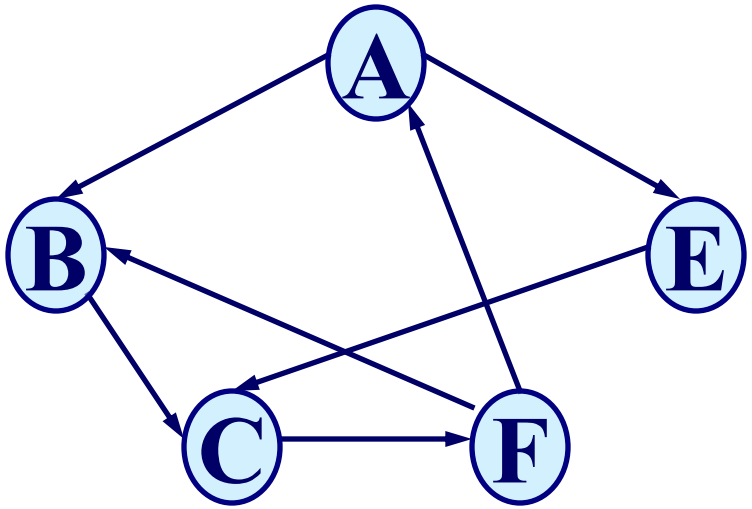
定义:矩阵的元素为

$$A_{ij}=\begin{cases}0 & (i,j)\notin VR\\ 1 & (i,j)\in VR\end{cases}$$



0	1	0	0	1	0
1	0	0	0	1	1
0	0	0	1	0	1
0	0	1	0	0	1
1	1	0	0	0	0
0	1	1	1	0	0

有向图的邻接矩阵为非对称矩阵



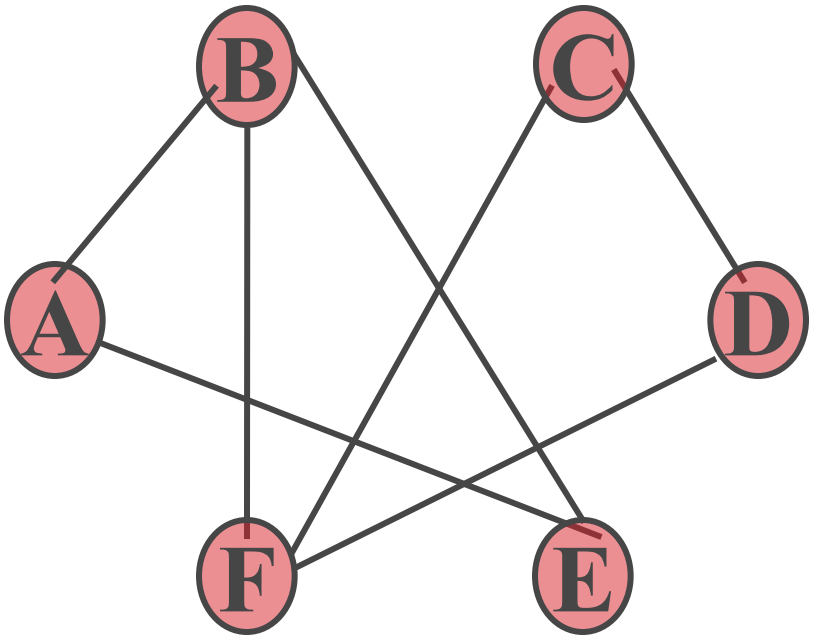
0	1	0	0	1
0	0	1	0	0
0	0	0	1	0
1	1	0	0	0
0	0	1	0	0

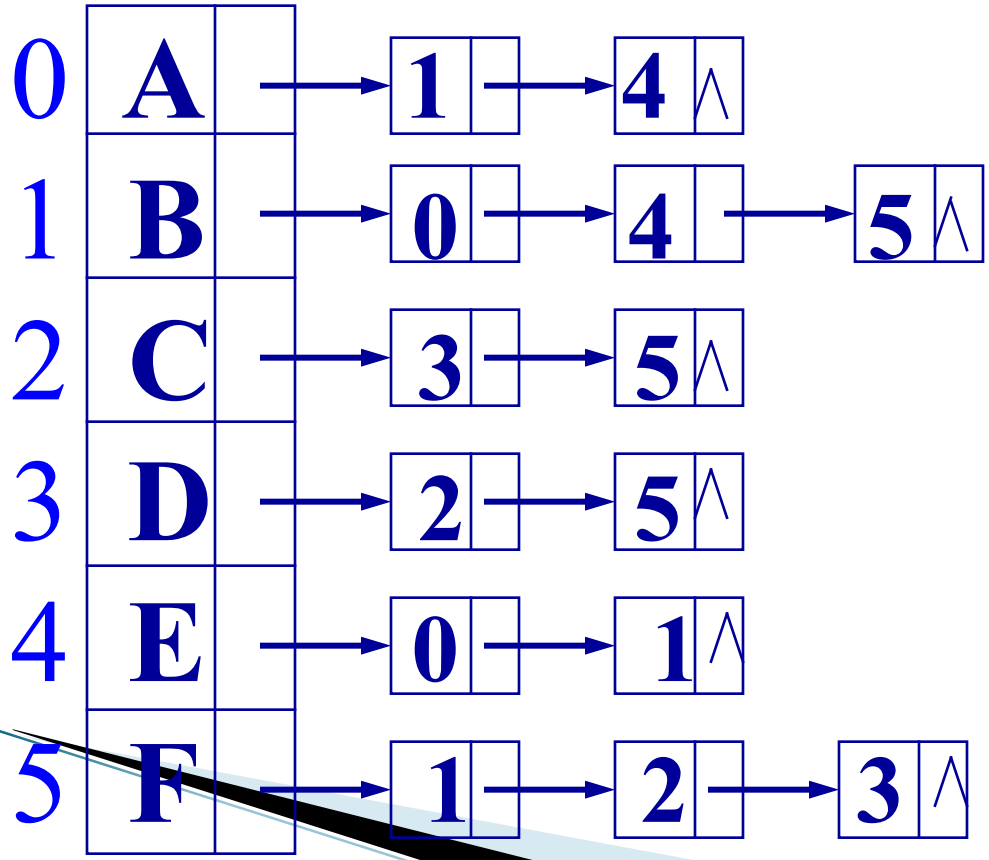
```
typedef struct ArcCell { // 弧的定义
  VRType  adj;    // VRType是顶点关系类型。
                  // 对无权图，用1或0表示相邻否；
                  // 对带权图，则为权值类型。
    InfoType  *info;  // 该弧相关信息的指针
} ArcCell,
    AdjMatrix[MAX_VERTEX_NUM]
                    [MAX_VERTEX_NUM];
```

```
typedef struct { // 图的定义
    VertexType       // 顶点信息
                vexs[MAX_VERTEX_NUM];
    AdjMatrix    arcs;      // 弧的信息
     int    vexnum, arcnum;   // 顶点数，弧数
     GraphKind   kind;     // 图的种类标志
} MGraph;
```

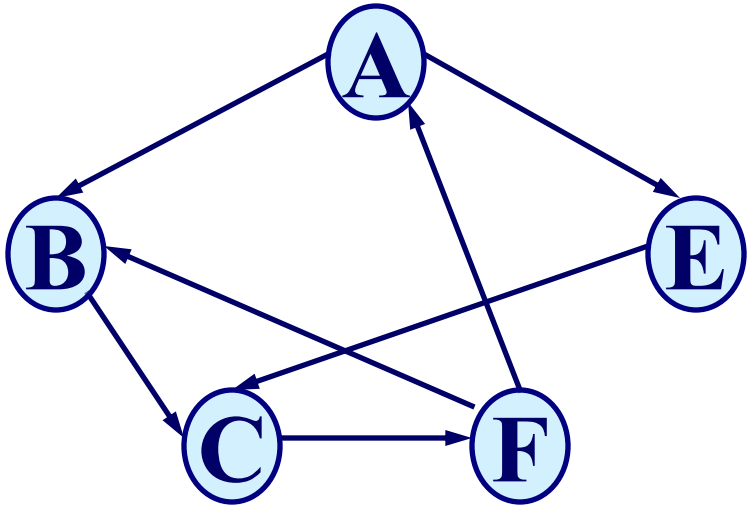


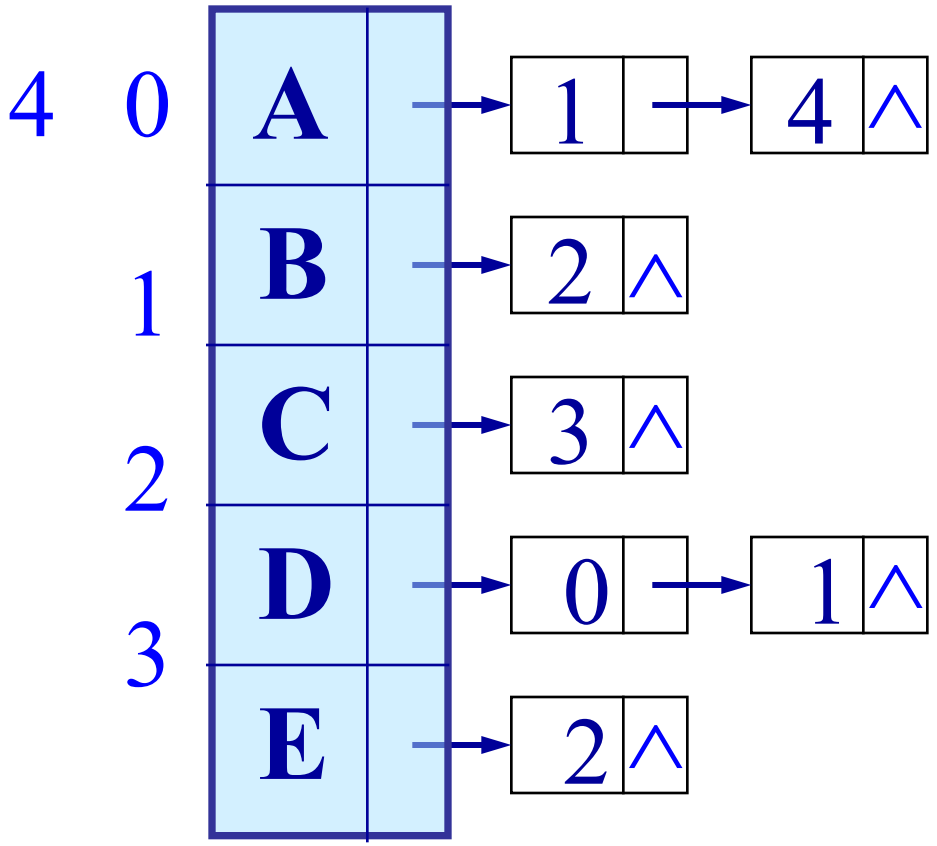
二、图的邻接表存储表示





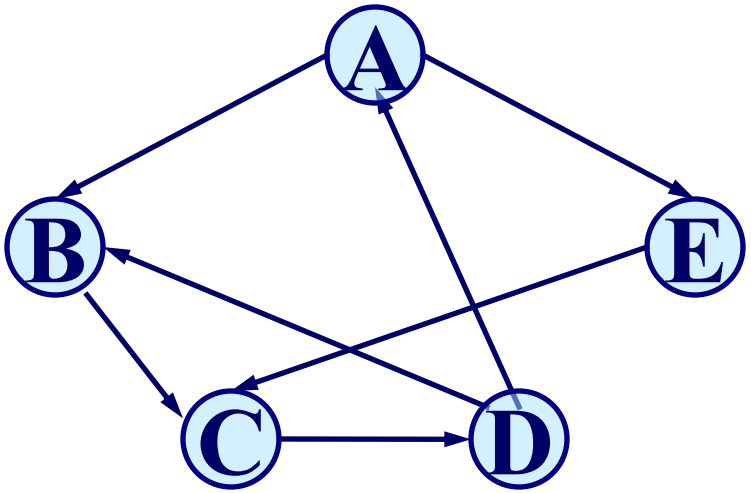
有向图的邻接表



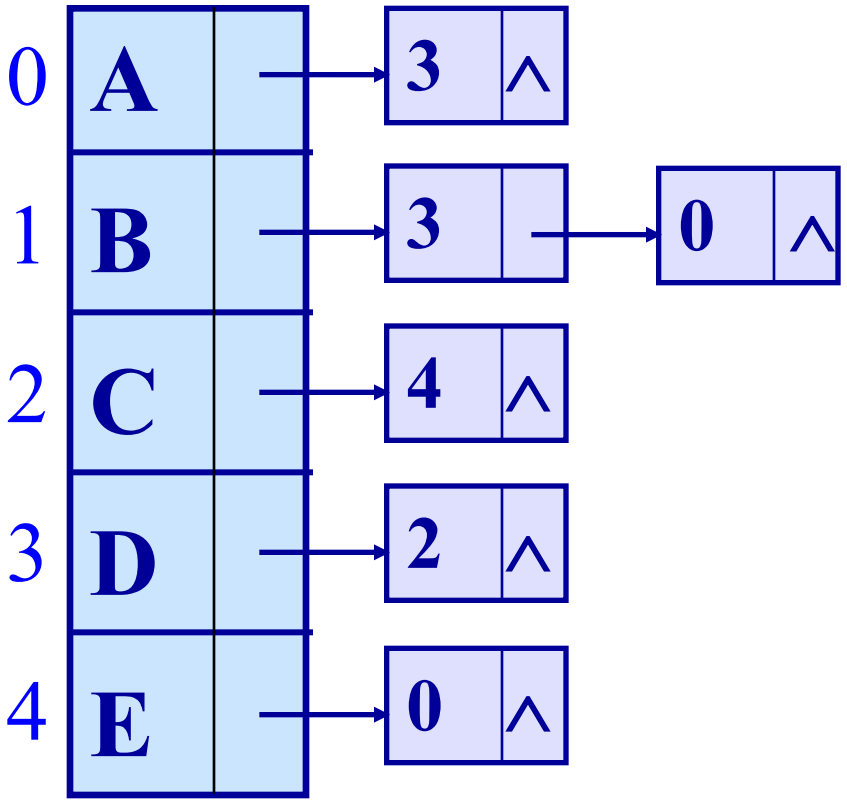


可见，在有向图的邻接表中不易找到指向该顶点的弧。

有向图的逆邻接表



在有向图的邻接表中，对每个顶点，链接的是指向该顶点的弧。

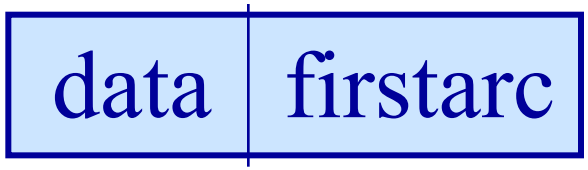


弧的结点结构




```
typedef struct ArcNode {
  int        adjvex;   // 该弧所指向的顶点的位置
  struct ArcNode  *nextarc;
                       // 指向下一条弧的指针
  InfoType   *info;   // 该弧相关信息的指针
} ArcNode;
```

顶点的结点结构




```
typedef struct VNode {
  VertexType  data;   // 顶点信息
  ArcNode  *firstarc;
                  // 指向第一条依附该顶点的弧
} VNode, AdjList[MAX_VERTEX_NUM];
```

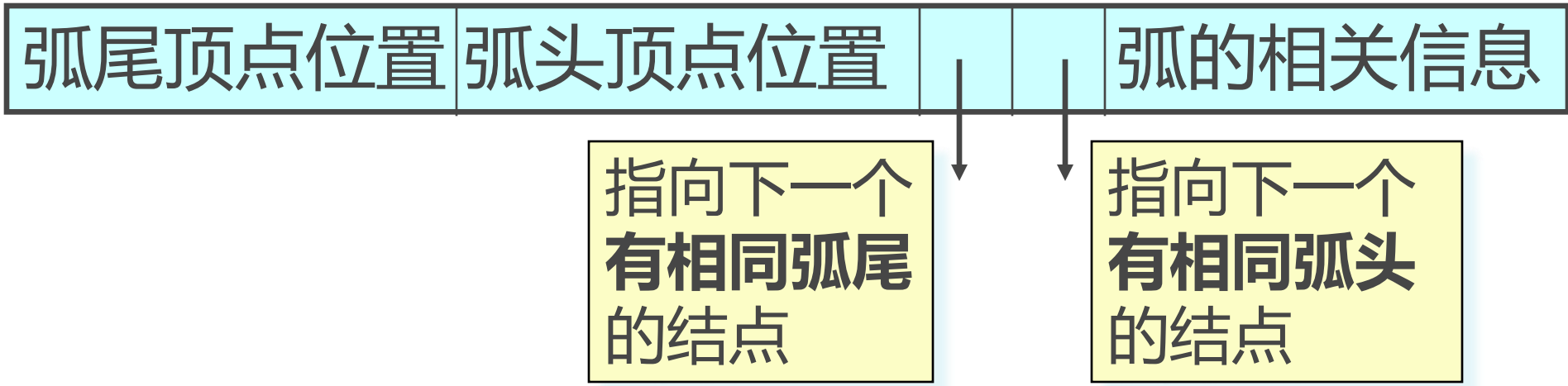
图的结构定义

```
typedef struct {
    AdjList  vertices;
    int      vexnum, arcnum;
    int      kind;          // 图的种类标志
} ALGraph;
```



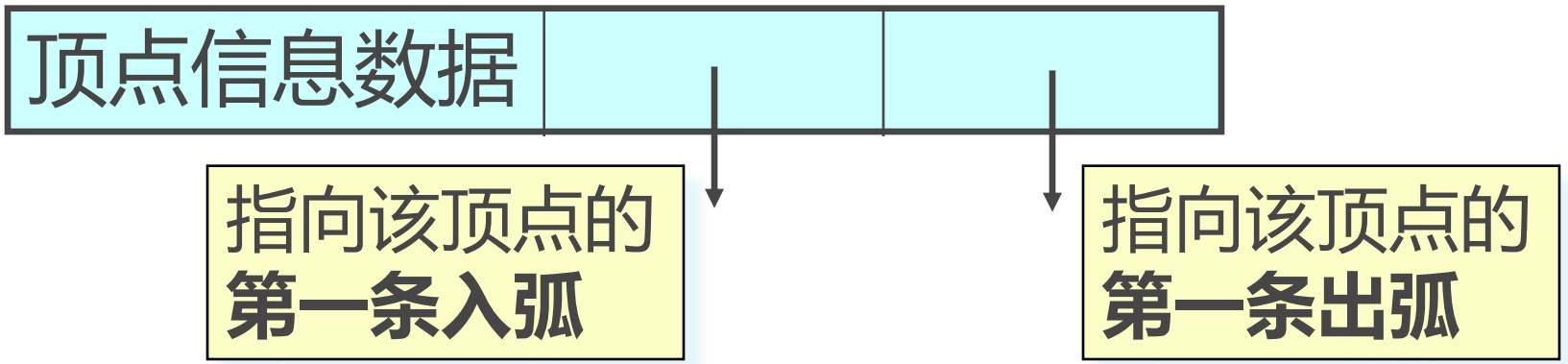
三、有向图的十字链表存储表示

弧的结点结构




```
typedef struct ArcBox { // 弧的结构表示
    int tailvex, headvex;   InfoType  *info;
    struct ArcBox  *hlink, *tlink;
    } VexNode;
```

顶点的结点结构




```
typedef struct VexNode { // 顶点的结构表示
    VertexType  data;
    ArcBox   *firstin, *firstout;
} VexNode;
```

有向图的结构表示(十字链表)

```
typedef struct {
   VexNode  xlist[MAX_VERTEX_NUM];
            // 顶点结点(表头向量)
   int    vexnum, arcnum;
            //有向图的当前顶点数和弧数
} OLGraph;
```



四、无向图的邻接多重表存储表示

边的结构表示

```
typedef struct Ebox {
    VisitIf      mark;      // 访问标记
    int      ivex, jvex;
                //该边依附的两个顶点的位置
    struct EBox  *ilink, *jlink;
    InfoType     *info;          // 该边信息指针
} EBox;
```

顶点的结构表示

```
typedef struct VexBox {
   VertexType  data;
   EBox  *firstedge; // 指向第一条依附该顶点的边
} VexBox;
```

无向图的结构表示

```
typedef struct {  // 邻接多重表
    VexBox  adjmulist[MAX_VERTEX_NUM];
    int   vexnum, edgenum;
} AMLGraph;
```



图的创建基本过程

- 输入图的类型，根据类型选择相应的创建算法
- 输入图的顶点数，边/弧数
- 输入并存储顶点信息
- 输入边/弧所关联的顶点对，将边或弧的信息存储到邻接矩阵/邻接表中

- 图的存储结构不同、图的类型不同，都会影响创建算法的实现细节；但是，图的总体创建流程是一致的。

```
Status CreateDG(Graph &G){//有向网
scanf (&G.vexnum, &G.arcnum, &IncInfo)
for ( i=0; i<G.vexnum ; i++)          scanf ( &G.vexs[i] );
for ( i = 0; i < G.vexnum ; i++)
    for ( j = 0; j < G.vexnum ; j++)
        G.arcs[i][j] = {INFINITY, NULL}; // 无弧位置先记为无穷大
for ( k = 0; k < G.arcnum ; k++){
    scanf (&v1, &v2, &w);
  i = LocateVex(G, v1); j = LocateVex(G, v2);  G.arcs[i][j].adj = w;
    if ( IncInfo) Input( *G.arcs[i][j].info);
```

7.3 图的遍历

从图中某个顶点出发游历图，访遍图中其余顶点，并且使图中的每个顶点仅被访问一次的过程。

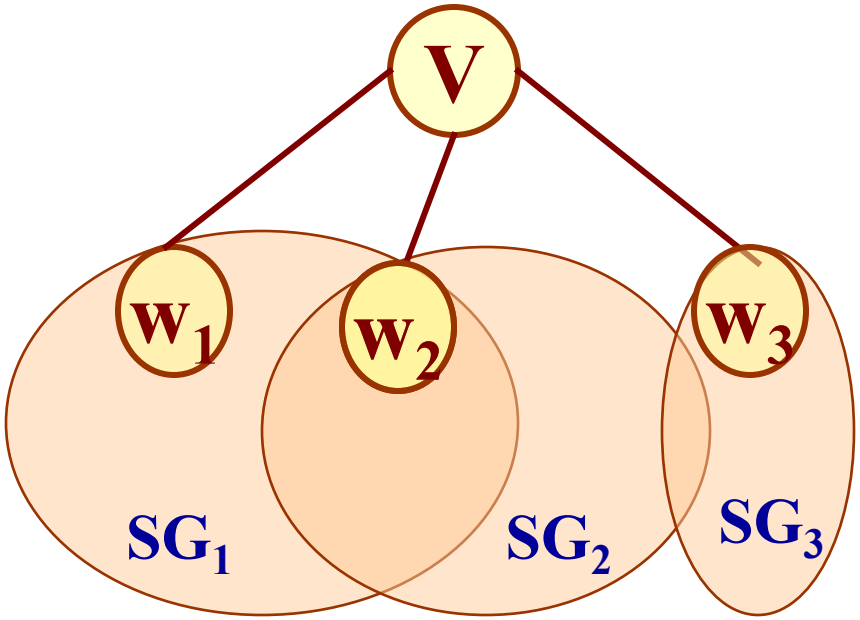
- **深度优先搜索**
- **广度优先搜索**
- **遍历应用举例**



一、深度优先搜索遍历图

连通图的深度优先搜索遍历

从图中某个顶点V_0出发，访问此顶点，然后**依次从V_0的各个未被访问的邻接点出发深度优先搜索遍历图**，直至图中所有和V_0有路径相通的顶点都被访问到。



W_1、W_2和W_3 均为 V 的邻接点，SG_1、SG_2 和 SG_3 分别为含顶点W_1、W_2和W_3 的子图。

访问顶点 V：

for (W_1、W_2、W_3)

若该邻接点W未被访问，

则从它出发进行深度优先搜索遍历。

从上页的图解可见:

1. 从深度优先搜索遍历连通图的过程类似于树的先根遍历;

2. 如何判别V的邻接点是否被访问?

解决的办法是:为每个顶点设立一个 “访问标志 visited[w]”。

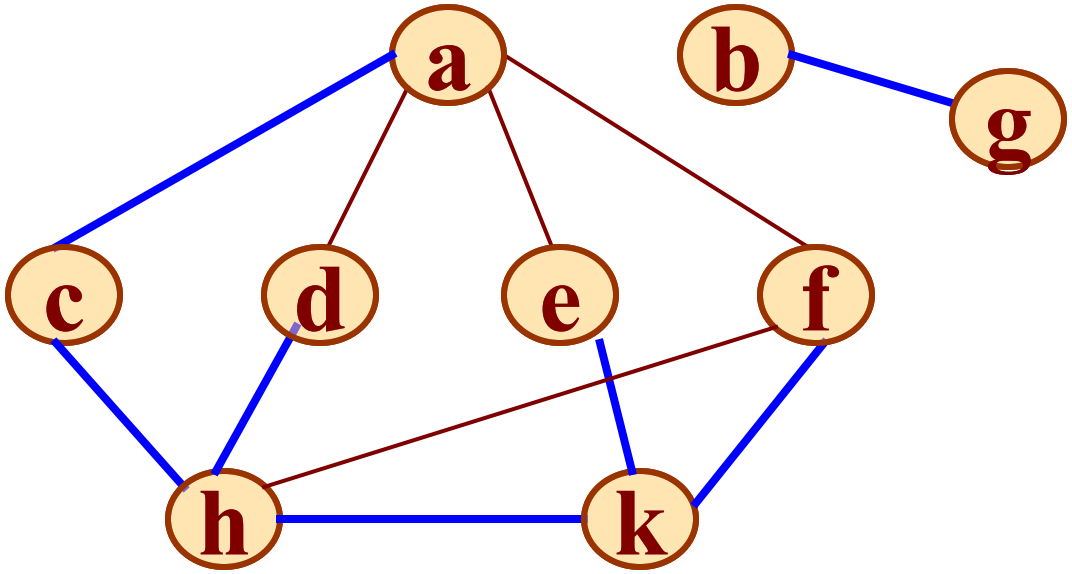
```
void DFS(Graph G, int v) {
  // 从顶点v出发，深度优先搜索遍历连通图 G
   visited[v] = TRUE;   VisitFunc(v); // 先标记，防止沿环重复访问
   for(w=FirstAdjVex(G, v);
         w!=0; w=NextAdjVex(G,v,w))
      if (!visited[w])  DFS(G, w);
         // 对v的尚未访问的邻接顶点w
         // 递归调用DFS
} // DFS
```

非连通图的深度优先搜索遍历

首先将图中每个顶点的访问标志设为 FALSE，之后搜索图中每个顶点，如果未被访问，则以该顶点为起始点，进行深度优先搜索遍历，否则继续检查下一顶点。

```
void DFSTraverse(Graph G,
                 Status (*Visit)(int v)) {
  // 对图 G 作深度优先遍历。
  VisitFunc = Visit;
  for (v=0; v<G.vexnum; ++v)
    visited[v] = FALSE; // 访问标志数组初始化
  for (v=0; v<G.vexnum; ++v)
    if (!visited[v])  DFS(G, v);
          // 对尚未访问的顶点调用DFS
}
```

例如:



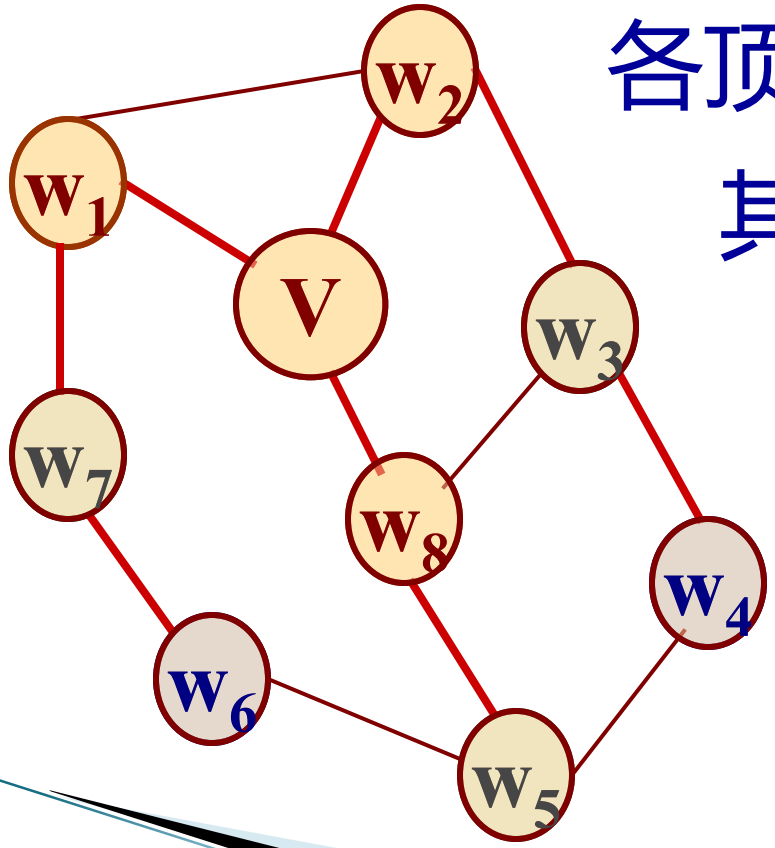
	0	1	2	3	4	5	6	7	8
访问标志:	T	T	T	T	T	T	T	T	T

访问次序:	a	c	h	d	k	f	e	b	g

邻接表为存储结构的图进行深度优先遍历

```
void DFSAL(ALGraph *G, int i)
  {/*以Vi为出发点对邻接表存储的图G进行DFS搜索*/
   EdgeNode *p;
   printf(G->AdjList [i]. VertexType );/*访问顶点Vi*/
   visited[i]=TRUE;                    /*标记Vi已访问*/
   p=G-> AdjList [i]. firstarc;        /*取Vi边表的头指针*/
   while(p)        /*依次搜索Vi的邻接点Vj，j=p-> adjvex */
  {if (!visited[p->adjvex])
     /*若Vj尚未访问，则以Vj为出发点向纵深搜索*/
     DFSAL(G,p->adjvex);
     p=p->nextarc;                     /*找Vi的下一个邻接点*/
```

二、广度优先搜索遍历图



对连通图，从起始点V到其余各顶点必定存在路径。

其中，$V->w_1$, $V->w_2$, $V->w_8$ 的路径长度为1；

$V->w_7$, $V->w_3$, $V->w_5$ 的路径长度为2；

$V->w_6$, $V->w_4$ 的路径长度为3。

从图中的某个顶点V_0出发，并在访问此顶点之后依次访问V_0的所有**未被访问**过的邻接点，之后**按这些顶点被访问的先后次序依次访问它们的邻接点**，直至图中所有和V_0有路径相通的顶点都被访问到。

若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点，重复上述过程，直至图中所有顶点都被访问到为止。

```
void BFSTraverse(Graph G,
                 Status (*Visit)(int v)){
  for (v=0; v<G.vexnum; ++v)
      visited[v] = FALSE;  //初始化访问标志
  InitQueue(Q);        // 置空的辅助队列Q
  for ( v=0;  v<G.vexnum;  ++v )
    if ( !visited[v]) {          // v 尚未访问
          …… ……
    }
} // BFSTraverse
```



```
visited[v] = TRUE;  Visit(v);    // 访问v
EnQueue(Q, v);                   // v入队列
while (!QueueEmpty(Q))  {
   DeQueue(Q, u);          // 队头元素出队并置为u
   for(w=FirstAdjVex(G, u); w!=0;
                      w=NextAdjVex(G,u,w))
      if ( ! visited[w])  {
         visited[w]=TRUE;  Visit(w); // 入队前标记，避免重复入队
         EnQueue(Q, w); // 访问的顶点w入队列
      } // if
} // while
```

邻接矩阵为存储结构的图广度优先遍历

```
for (j=0;j<G->vexnum;j++)
  /*依次搜索Vi的邻接点Vj*/
      if (G->edges[u][j]==1 && !visited[j])
       /*若Vj未访问*/
       {visited[j]=TRUE;
        printf(G->vexs[j]); /*访问Vj */
        EnQueue(Q,j); /*访问过的Vj入队列*/
       }
```

三、遍历应用举例

1. 求一条从顶点 i 到顶点 s 的简单路径

2. 求两个顶点之间的一条路径长度最短的路径



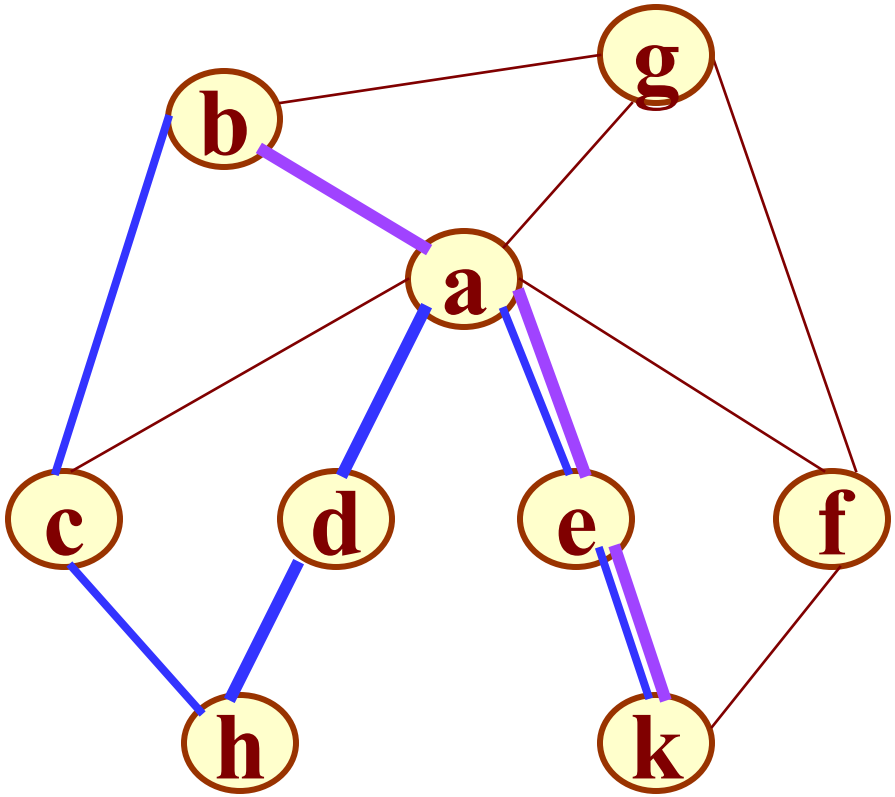
1. 求一条从顶点 i 到顶点 s 的简单路径

例如：求从顶点 b 到顶点 k 的一条简单路径。

从顶点 b 出发进行深度优先搜索遍历。

假设找到的第一个邻接点是c，则得到的结点访问序列为：

b c h d a e k f g,



假设找到的第一个邻接点是a，则得到的结点访问序列为：b a d h c e k f g。

结论:

1. 从顶点 i 到顶点 s，若存在路径，则从顶点 i 出发进行深度优先搜索，必能搜索到顶点 s。

2. 遍历过程中搜索到的顶点不一定是路径上的顶点。

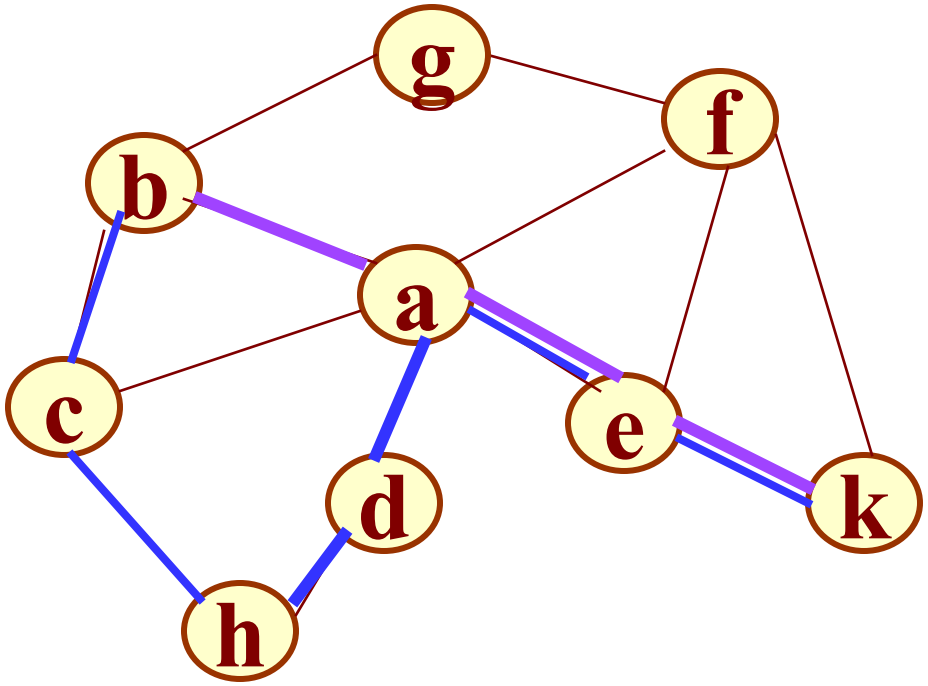
3. 由它出发进行的深度优先遍历已经完成的顶点不是路径上的顶点。

```
void DFSearch( int v, int s, char *PATH)  {
 // 从第v个顶点出发递归地深度优先遍历图G，
 // 求得一条从v到s的简单路径，并记录在PATH中
  visited[v] = TRUE;   // 访问第 v 个顶点
  Append(PATH, getVertex(v));   // 第v个顶点加入路径
  for (w=FirstAdjVex(v);  w!=0&&!found;
                          w=NextAdjVex(v) )
     if (w=s) { found = TRUE;  Append(PATH, w); }
     else  if (!visited[w])  DFSearch(w, s, PATH);
  if (!found)  Delete (PATH); // 从路径上删除顶点 v
}
```



2. 求两个顶点之间的一条路径长度最短的路径

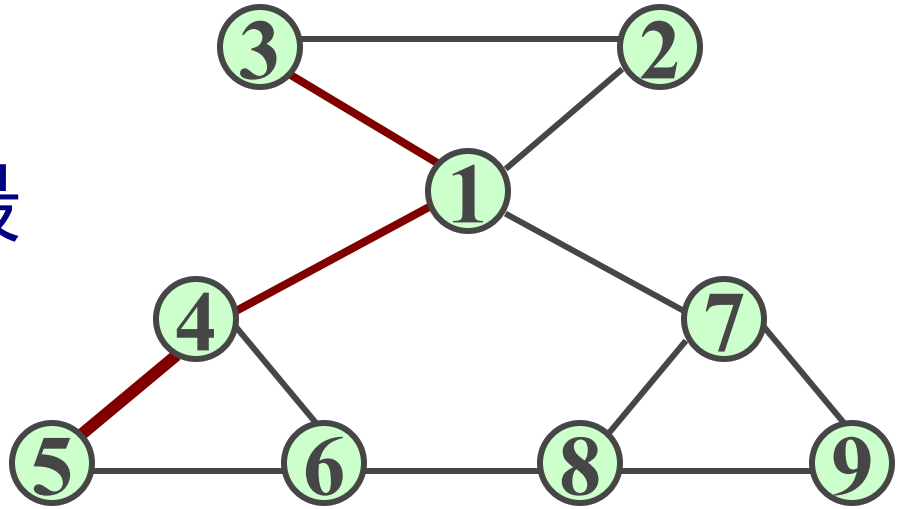
若两个顶点之间存在多条路径，则其中必有一条路径长度最短的路径。如何求得这条路径?



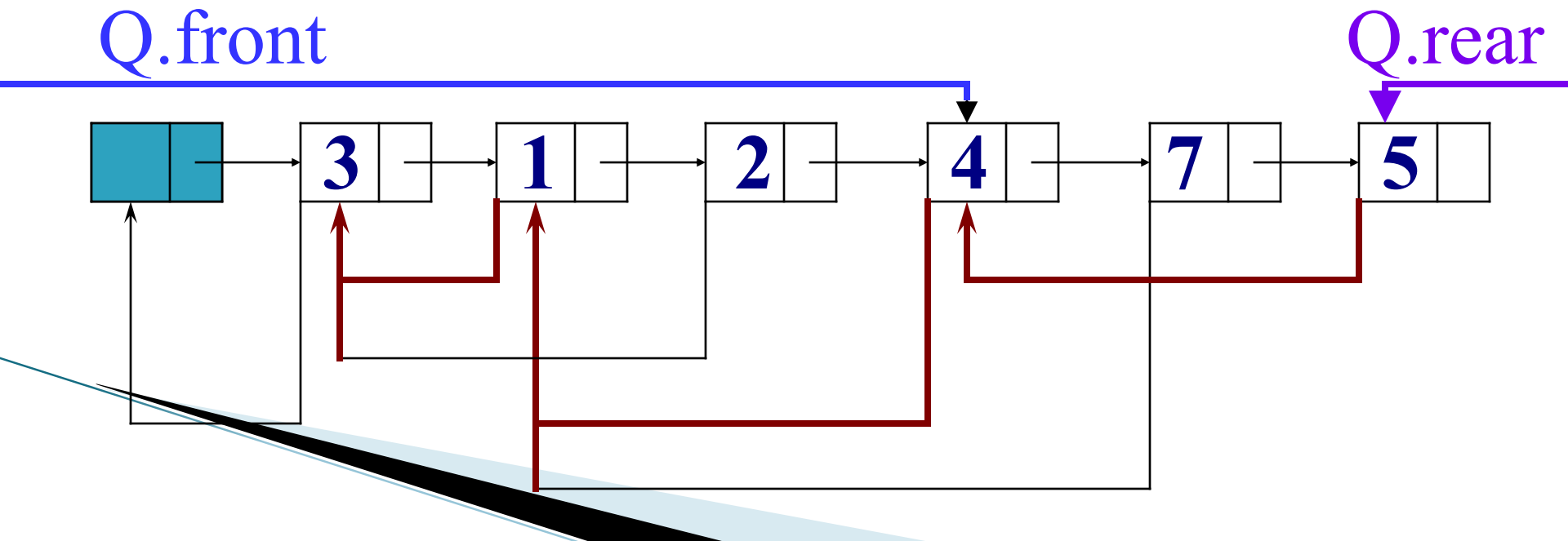
深度优先搜索访问顶点的**次序**取决于图的**存储结构**，而广度优先搜索访问顶点的次序是按“路径长度”渐增的次序。

因此，**求路径长度最短的路径可以基于广度优先搜索遍历进行**，但需要**修改链队列的结点结构及其入队列和出队列的算法。**

例如:求下图中顶点 3 至顶点 5 的一条最短路径。



链队列的状态如下所示:



1) **将链队列的结点改为“双链”结点。**即结点中包含next 和priou两个指针；

2) **修改入队列的操作。**插入新的队尾结点时，令其priou域的指针指向刚刚出队列的结点，即当前的队头指针所指结点；

3) **修改出队列的操作。**出队列时，仅移动队头指针，而不将队头结点从链表中删除。

```
typedef  DuLinkList QueuePtr;
void InitQueue(LinkQueue& Q) {
  Q.front=Q.rear=(QueuePtr)malloc(sizeof(QNode));
  Q.front->next = Q.rear->next = NULL
}
void EnQueue( LinkQueue& Q, QelemType e ) {
  p = (QueuePtr) malloc (sizeof(QNode));
  p->data = e;  p->next = NULL;
  p->priou = Q.front
  Q.rear->next = p;  Q.rear = p;
}
void DeQueue( LinkQueue& Q, QelemType& e ) {
  Q.front = Q.front->next;  e = Q.front->data
}
```

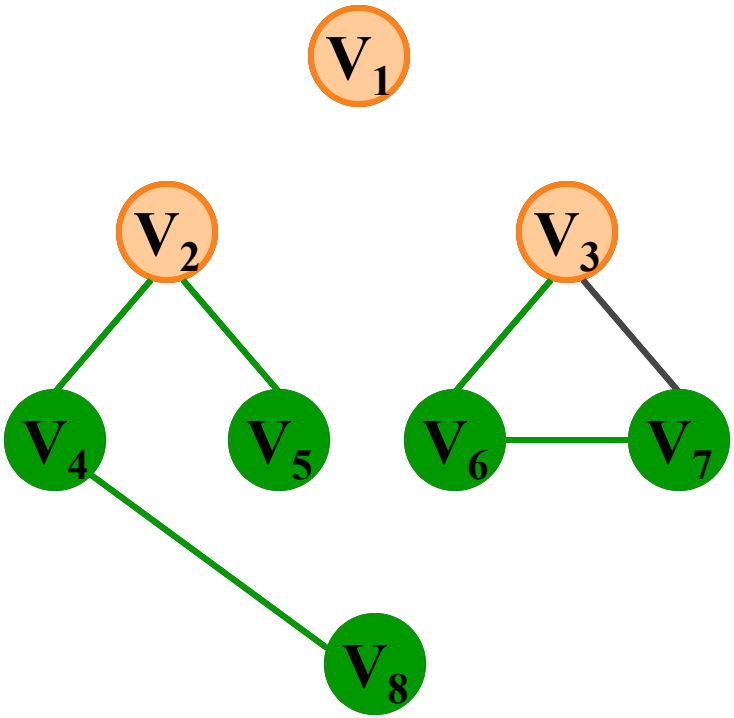


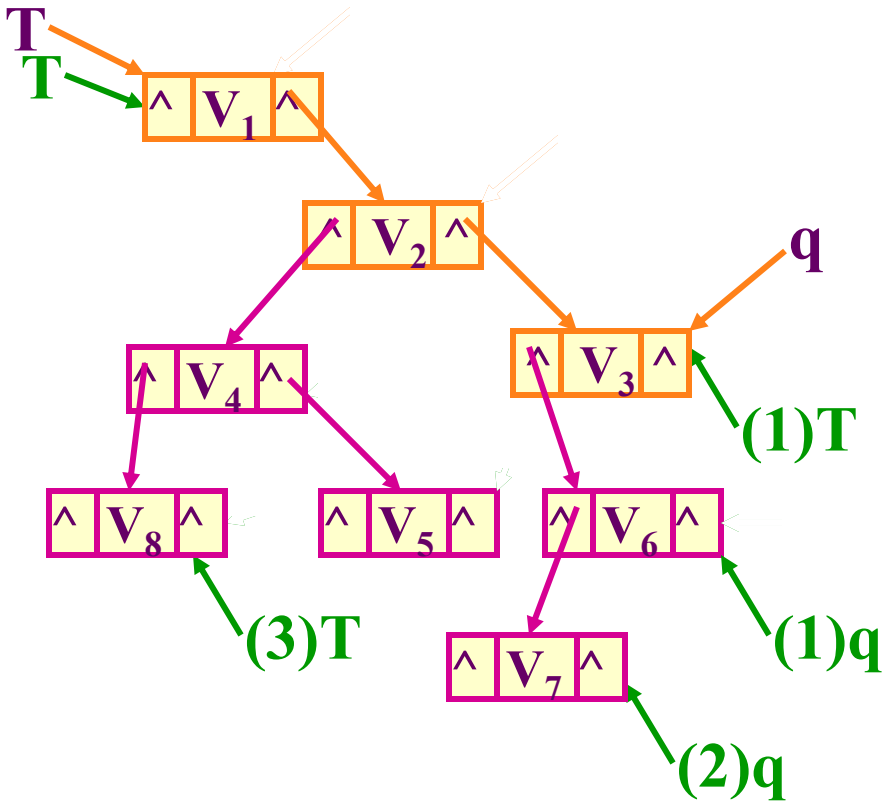
3 图的生成树

设图G=(V, E)是个连通图, 当从图任一顶点出发遍历图G时, 将边集E(G)分成两个集合A(G)和B(G)。其中A(G)是遍历图时所经过的边的集合, B(G)是遍历图时未经过的边的集合。显然, G1=(V, A)是图G的子图。我们称子图G1是连通图G的生成树

算法思路

也是图的遍历算法的应用.
将访问顶点变成创建生成树或增加一个结点到生成树中的操作.
生成树/森林的存储表示采用孩子-兄弟表示法.


V1
V2
V3
V4
V5
V6
V7
V8


T
T
^ V1 ^
^ V2 ^
q
^ V4 ^
^ V3 ^
(1)T
^ V8 ^
^ V5 ^
^ V6 ^
(3)T
(1)q
^ V7 ^
(2)q

深度优先搜索生成树/森林算法思路

```
void DFSForest(Graph G, CSTree &T){
T = NULL;
for ( v=0; v<G.vexnum; ++v ) visited[v] = FALSE;
for ( v=0; v<G.vexnum; ++v )
if ( !visited[v] ){
p = ( CSTree ) malloc(sizeof(CSNode));
*p = {GetVex(G, v), NULL, NULL};
if ( T == NULL ) T = p; /* 第一棵生成树的根*/
else   q->nextsbling = p;  /* 其它生成树的根*/
q = p;          /* q指示当前生成树的根 */
/* 建立以p指向的结点为根结点的生成树 */
DFSTree(G, v, p);}
}
```



```
void DFSTree(Graph G, int v, CSTree & T )
{  visited[v] = TRUE;
first = TRUE;/* 标识下一个访问的邻接点是否为第一个未访问的邻接点 */
for ( w = FirstAdjVex( G, v); w>=0; w = NextAdjVex(G, v, w) )
if ( !visited[w] ){
p = ( CSTree ) malloc(sizeof(CSNode));/* 分配孩子结点 */
p = {GetVex(G, w), NULL, NULL};
if ( first ){  /* w是v的第一个未访问的邻接点 */
T->firstchild = p; first = FALSE;/*是根的第一个孩子 */
} else {
q->nextsbling = p;  /* 是上一邻接点的右兄弟结点 */
}
q = p;   /* 修改上一孩子结点的指针 */
DFSTree(G, w, q);
}
}
```

7.4 (连通网的)最小生成树

问题：

假设要在 n 个城市之间建立通讯联络网，则连通 n 个城市只需要修建 $n-1$ 条线路，**如何在最节省经费的前提下建立**这个**通讯网**？



该问题等价于：

构造网的一棵最小生成树，即：

在 e 条带权的边中选取 n-1 条边（不构成回路），使“**权值之和**”为最小。

算法一：（普里姆算法）

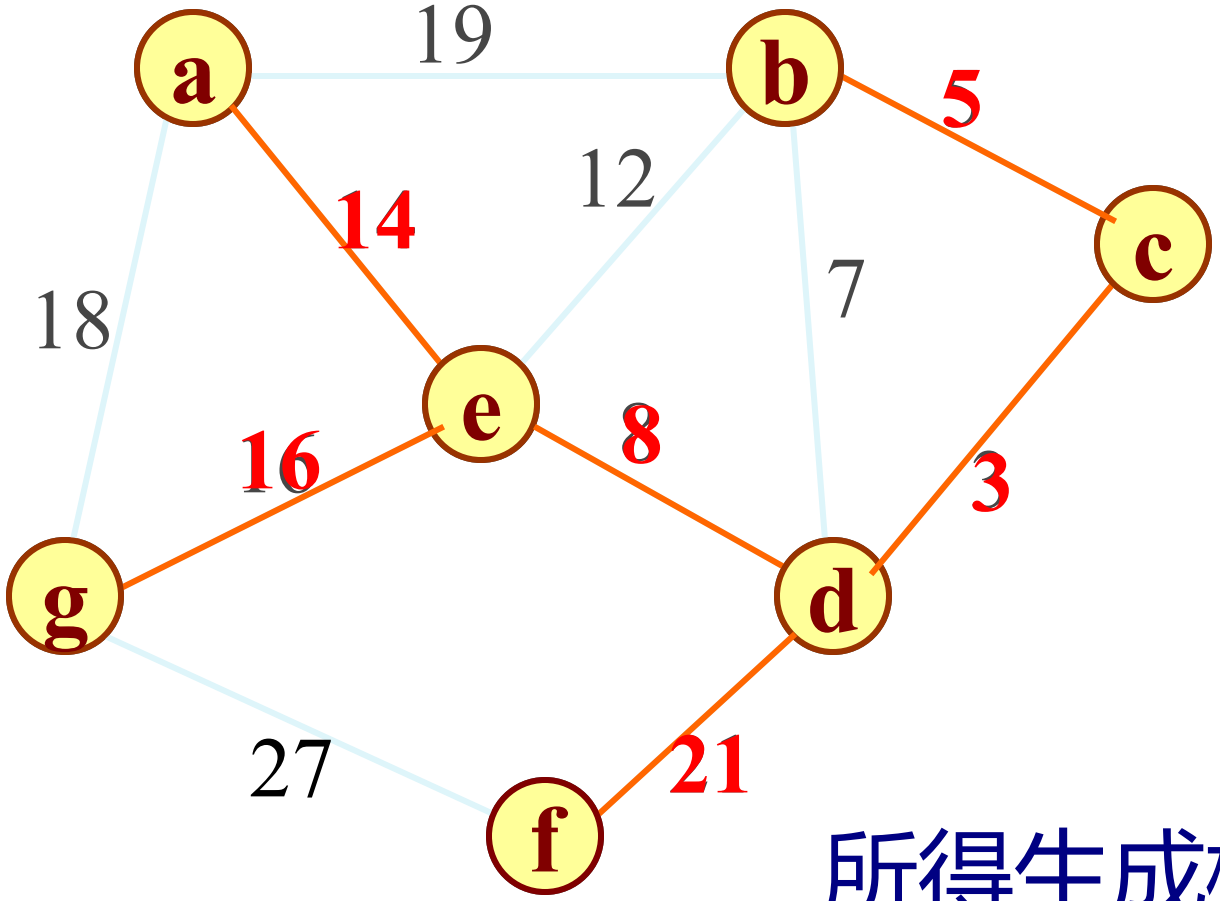
算法二：（克鲁斯卡尔算法）



普里姆算法的基本思想:

取图中任意一个顶点 v 作为生成树的根，之后往生成树上添加新的顶点 w。**在添加的顶点 w 和已经在生成树上的顶点v 之间必定存在一条边，并且该边的权值在所有连通顶点 v 和 w 之间的边中取值最小。**之后继续往生成树上添加顶点，直至生成树上含有 n-1 个顶点为止。

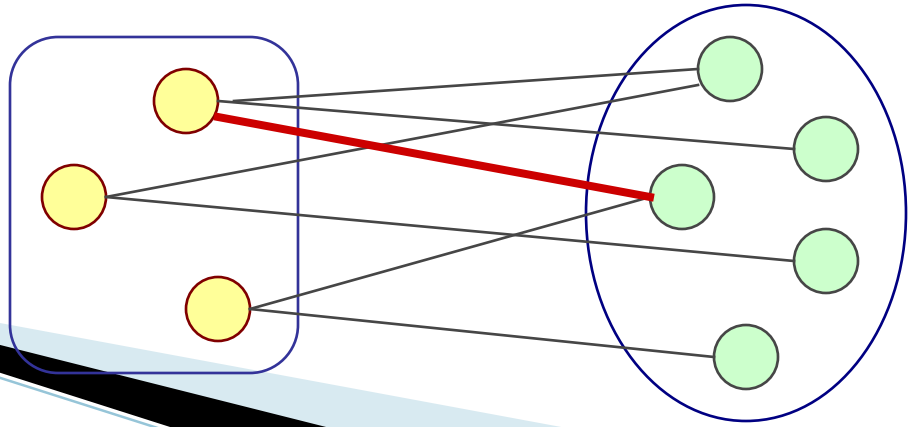
例如:



所得生成树权值和

= 14+8+3+5+16+21 = 67

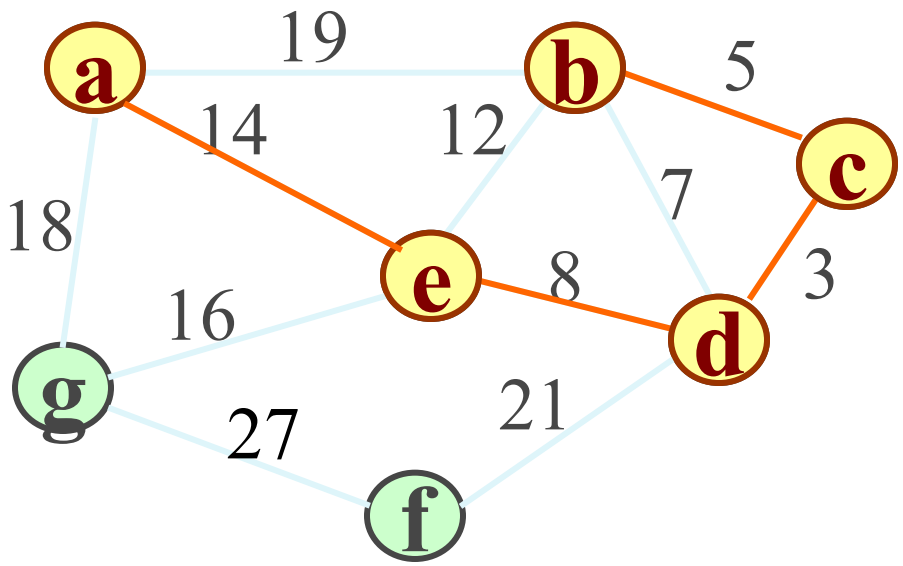
一般情况下所添加的顶点应满足下列条件：在生成树的构造过程中，图中 n 个顶点分属两个集合：已落在生成树上的顶点集 U 和尚未落在生成树上的顶点集 V-U，则应**在所有连通U中顶点和V-U中顶点的边中选取权值最小的边。**



设置一个辅助数组，对当前V - U集中的每个顶点，记录和顶点集U中顶点相连接的代价最小的边：

```
struct {
    VertexType  adjvex;  // U集中的顶点序号
    VRType      lowcost;  // 边的权值
} closedge[MAX_VERTEX_NUM];
```

例如：



closedge	0	1	2	3	4	5	6
Adjvex		c	d	e	a	d	e
Lowcost		**5**	**3**	**8**	**14**	21	16

```
void MiniSpanTree_P(MGraph G, VertexType u) {
 //用普里姆算法从顶点u出发构造网G的最小生成树
  k = LocateVex ( G, u );
  for ( j=0; j<G.vexnum; ++j )  // 辅助数组初始化
      if (j!=k)
           closedge[j] = { u, G.arcs[k][j].adj };
  closedge[k].lowcost = 0;      // 初始，U = {u}
  for (i=0; i<G.vexnum; ++i) {

      继续向生成树上添加顶点;
}
```



```
k = minimum(closedge); // 从横跨集合 U 的候选边中取最小值
                       // 求出加入生成树的下一个顶点(k)
printf(closedge[k].adjvex, G.vexs[k]);
                       // 输出生成树上一条边
closedge[k].lowcost = 0;    // 第k顶点并入U集
for (j=0; j<G.vexnum; ++j)
     //修改其它顶点的最小边
  if (G.arcs[k][j].adj < closedge[j].lowcost)
    closedge[j] = { G.vexs[k], G.arcs[k][j].adj };
```

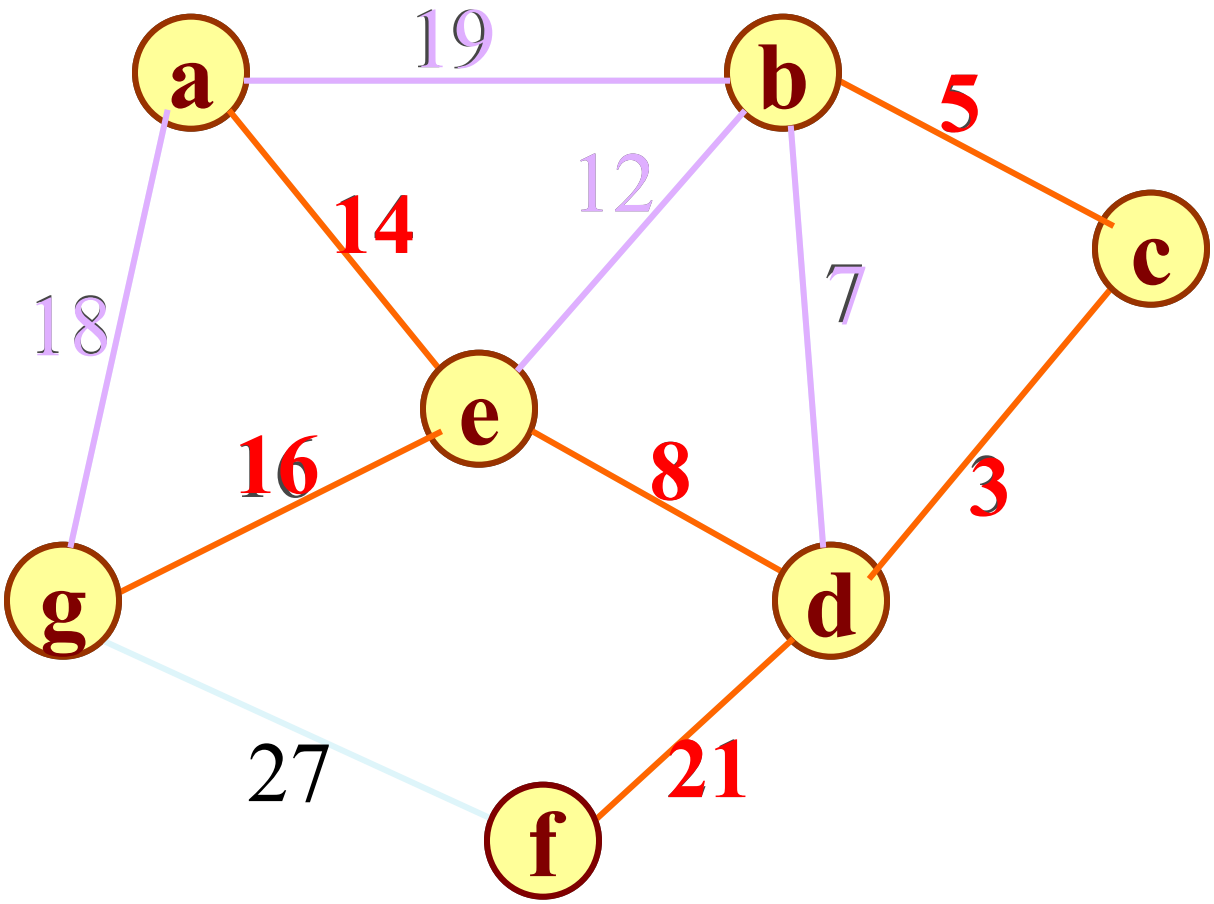


克鲁斯卡尔算法的基本思想：

- **考虑问题的出发点：**为使生成树上边的权值之和达到最小，则应使生成树中每一条边的权值尽可能地小。

- **具体做法：**先构造一个只含 n 个顶点的子图SG，然后从权值最小的边开始，若它的添加不使SG 中产生回路，则在 SG 上加上这条边，如此重复，直至加上 n-1 条边为止。

例如:



算法描述:

构造非连通图 ST=(V,{ });

k = i = 0;　// k 计选中的边数

while (k<n-1) {

++i;

检查边集 E 中**第 i 条权值最小的边**(u,v);

若(u,v)加入ST后不使ST中产生回路，

则 输出边(u,v); 且 k++;

}

比较两种算法

算法名	普里姆算法	克鲁斯卡尔算法
时间复杂度	$O(n^2)$	O(eloge)
适应范围	稠密图	稀疏图



克鲁斯卡尔算法

第 77–80 页给出选边规则、例图和伪代码。补充用顶点所属集合判定成环的代码。输入是按权值非递减排列的无向边，至少有一个顶点，顶点编号为 0..n-1。picked 可容纳 n-1 条边。返回值小于 n-1 时，图不连通。

```
typedef struct {
    int u, v, weight;
} Edge;

static int component_root(int parent[], int v)
{
    while (parent[v] != v) {
        parent[v] = parent[parent[v]];
        v = parent[v];
    }
    return v;
}

int kruskal(int n, const Edge edges[], int m, Edge picked[])
{
    if (n <= 1) {
        return 0;
    }
    int parent[n], size[n];
    for (int v = 0; v < n; ++v) {
        parent[v] = v;
        size[v] = 1;
    }
    int count = 0;
```

克鲁斯卡尔算法

```
    for (int i = 0; i < m && count < n - 1; ++i) {
        int a = component_root(parent, edges[i].u);
        int b = component_root(parent, edges[i].v);
        if (a == b) {
            continue;  // 两端已经连通，加入该边会成环
        }
        if (size[a] < size[b]) {
            int temp = a;
            a = b;
            b = temp;
        }
        parent[b] = a;
        size[a] += size[b];
        picked[count++] = edges[i];
    }
    return count;
}
```

7.5 重（双）连通图和关节点

问题：

若从一个连通图中删去任何一个顶点及其相关联的边，它仍为一个连通图的话，则该连通图被称为重（双）连通图。

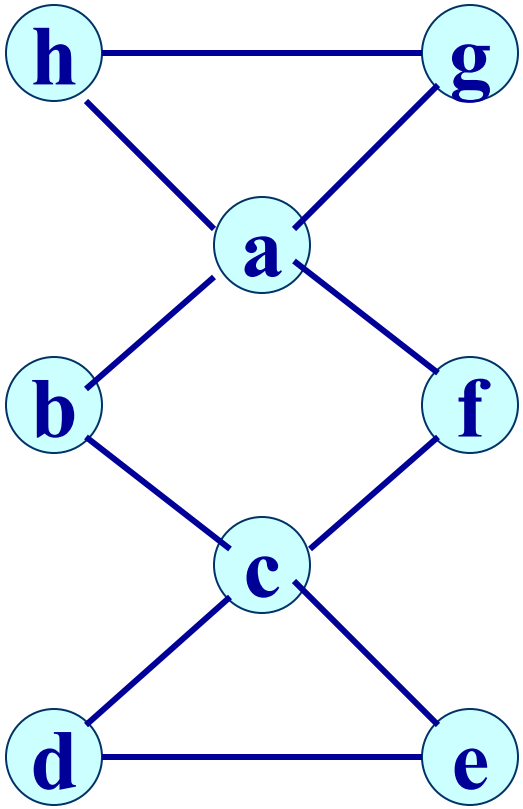
如何判别给定的连通图是否是双连通图?

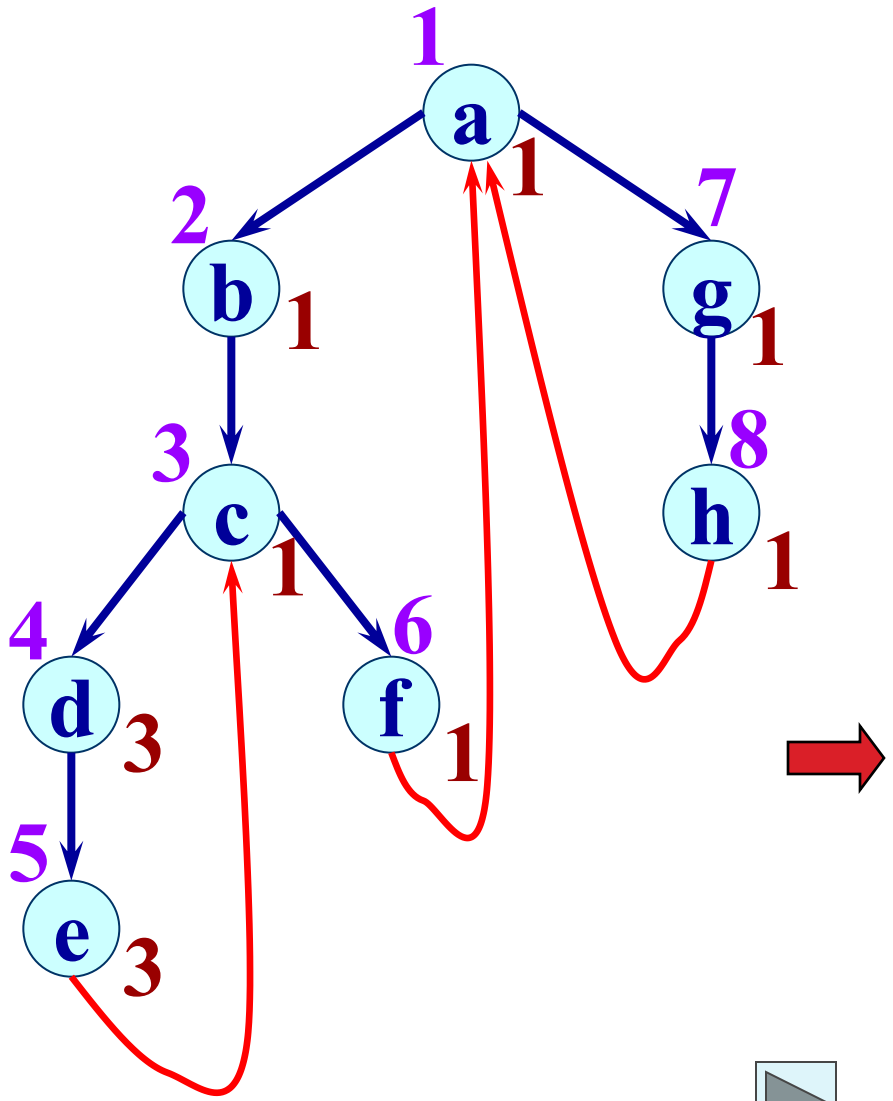
若连通图中的某个顶点和其相关联的边被删去之后，该连通图被分割成两个或两个以上的连通分量，则称此顶点为**关节点**。

没有关节点的连通图为双连通图。

例如:下列连通图中，

深度优先生成树





顶点 a 和顶点 c 是关节点

关节点有何特征?

需借助图的深度优先生成树来分析。

假设从某个顶点V_0出发对连通图进行深度优先搜索遍历，则可得到一棵**深度优先生成树**，树上包含图的所有顶点。



- 若生成树的根结点，有两个或两个以上的分支，则此顶点(生成树的根)必为关节点；
- 对生成树上的任意一个“顶点”，若其某棵子树的根或子树中的其它“顶点”没有和其祖先相通的回边，则该“顶点”必为关节点。

对上述两个判定准则在算法中如何实现?

1）设V_0为深度优先遍历的出发点

```
p = G.vertices[0].firstarc;   v = p->adjvex;
DFSArticul(G, v);
    // 从第v顶点出发深度优先搜索
if (count < G.vexnum) {
    // 生成树的根有至少两棵子树
   printf (0, G.vertices[0].data);
    // 根是关节点
```

2) 定义函数:

low(v) = Min{visited[v], low[w], visited[k] }

其中: 顶点**w** 是生成树上顶点**v** 的**孩子**;

顶点**k** 是生成树上和顶点**v**由**回边**相联接的**祖先**;

visited记录深度优先遍历时的访问次序

若对顶点v，在生成树上存在一个子树根w，

且 low[w] ≥ visited[v]

则顶点v为关节点。



对深度优先遍历算法作如下修改：

1．visited[v]的值改为遍历过程中顶点的访问次序count值；

2．遍历过程中求得
low[v]=**Min**{visited[v],low[w],visited[k]}

3．从子树遍历返回时，
判别low[w]≥visited[v]?

```
void  DFSArticul(ALGraph G, int v0) {
// 从第v0个顶点出发深度优先遍历图 G,
// 查找并输出关节点
 min =visited[v0] = ++count;
  // v0是第count个访问的顶点, 并设low[v0]的初值为min
  for(p=G.vertices[v0].firstarc;   p;
                                   p=p->nextarc) {
    // 检查v0的每个邻接点
  }
  low[v0] = min;

} // DFSArticul
```

```
    w = p->adjvex;       // w为v0的邻接顶点
    if (visited[w] == 0) {       // w未曾被访问
      DFSArticul(G, w);  // 返回前求得low[w]
if (low[w] < min)   min = low[w];
if (low[w]>=visited[v0]) // 该子树无法绕过 v0 到达其祖先
  printf(v0, G.vertices[v0].data); //输出关节点
    }
    else     // w是回边上的顶点
 if (visited[w] < min)   min = visited[w];
```



7.6 拓扑排序

问题:

假设以有向图表示一个工程的施工图或程序的数据流图，则图中不允许出现回路。

检查有向图中是否存在回路的方法之一，是对有向图进行**拓扑排序**。

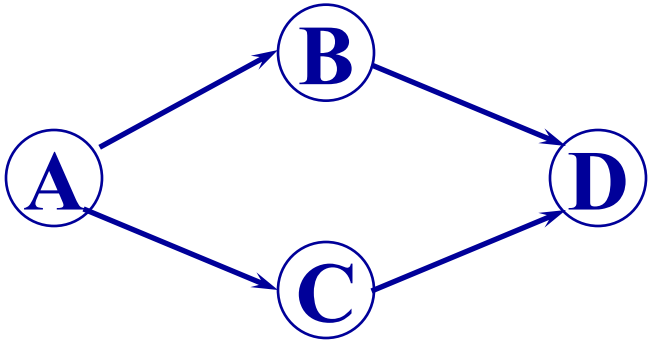
何谓“拓扑排序”？

对有向图进行如下操作：

按照有向图给出的次序关系，将图中顶点排成一个线性序列，对于有向图中没有限定次序关系的顶点，则可以人为加上任意的次序关系。

由此所得顶点的线性序列称之为拓扑有序序列

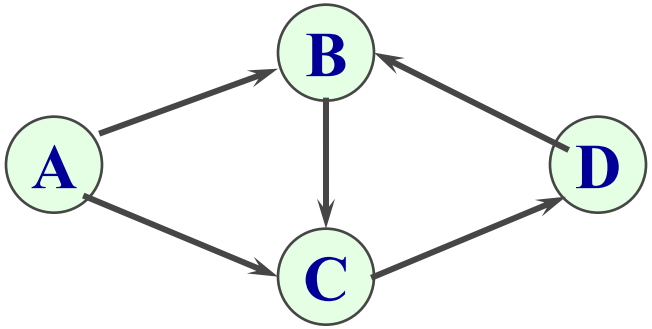
例如：对于下列有向图



可求得拓扑有序序列：

A B C D　或　A C B D

反之，对于下列有向图



不能求得它的拓扑有序序列。

因为图中存在一个回路 {B, C, D}

如何进行拓扑排序?

一、从有向图中选取一个**没有前驱**的顶点，并输出之；

二、从有向图中**删去此顶点以及所有以它为尾的弧**；

重复上述两步，直至图空，或者图不空但找不到无前驱的顶点为止。

a b h c d g f e

在算法中需要用定量的描述替代定性的概念

没有前驱的顶点 == 入度为零的顶点

删除顶点及以它为尾的弧 == 弧头顶点的入度减1

算法描述

```
取入度为零的顶点v;
while (v<>0) {
    printf(v);  ++m;
    w:=FirstAdj(v);
    while (w<>0) {
      inDegree[w]--;
      w:=nextAdj(v,w);
    }
    取下一个入度为零的顶点v;
}
if m<n  printf("图中有回路" );
```

为避免每次都要搜索入度为零的顶点，在算法中设置一个**“栈”**，以保存“入度为零”的顶点。

```
CountInDegree(G,indegree);
        //对各顶点求入度
InitStack(S);
for ( i=0; i<G.vexnum; ++i)
   if (!indegree[i])  Push(S, i);
        //入度为零的顶点入栈
```

```
count=0;          //对输出顶点计数
while (!EmptyStack(S)) {
  Pop(S, v);  ++count;  printf(v);
  for (w=FirstAdj(v); w;  w=NextAdj(G,v,w)){
    --indegree(w);  // 弧头顶点的入度减一
    if (!indegree[w])  Push(S, w);
          //新产生的入度为零的顶点入栈
  }
}
if (count<G.vexnum) printf("图中有回路" )
```



- 栈在这里的作用只是起到一个保存当前入度为零点的顶点，并使之处理有序。这种有序可以是后进先出，也可以是先进先出，故此也可用队列来辅助实现。

```
ptr=G->adjlist[V].firstedge;
while (ptr!=null)
   { w=ptr->adjvex;
        --indegree(w);
       if (!indegree[w])  Push(S, w);
     ptr=ptr->next;
   }
```

AOE网

若在带权的有向图中，以顶点表示事件，以有向边表示活动，边上的权值表示活动的开销（如该活动持续的时间），则此带权的有向图称为AOE网。

AOE网具有以下两个性质：

①只有在某顶点所代表的事件发生后，从该顶点出发的各有向边所代表的活动才能开始。

②只有在进入一某顶点的各有向边所代表的活动都已经结束，该顶点所代表的事件才能发生。。

关键路径

问题:

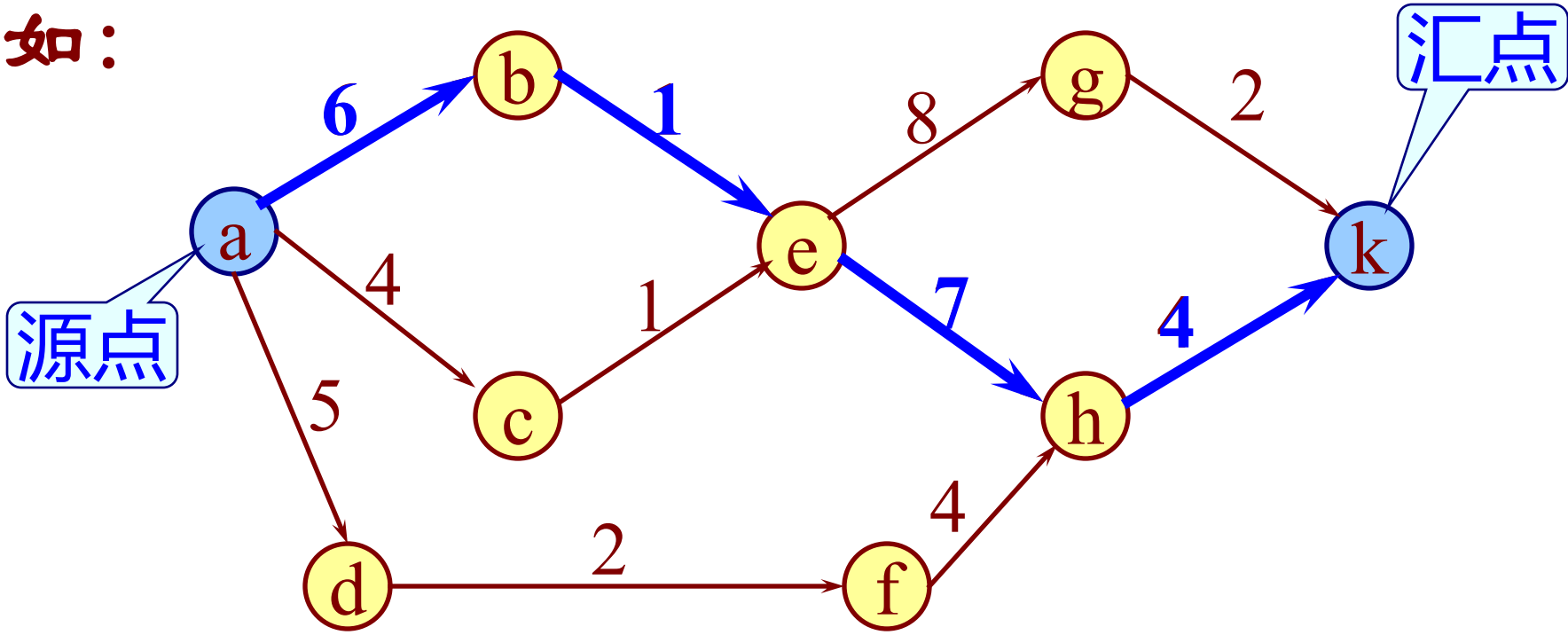
假设以有向网表示一个施工流图，弧上的权值表示完成该项子工程所需时间。

问：哪些子工程项是“关键工程”？

即：哪些子工程项将影响整个工程的完成期限的。

整个工程完成的时间为：从有向图的**源点**到**汇点**的最长路径。

例如：



“关键活动”指的是：该弧上的**权值增加**将使有向图上的**最长路径的长度增加。**

如何求关键活动?

“事件(顶点)” 的 最早发生时间 $ve(j)$

ve(j) = 从源点到顶点j的最长路径长度;

“事件(顶点)” 的 最迟发生时间 $vl(k)$

vl[k]是指在不推迟整个工期的前提下，事件vk允许的最晚发生时间。

vl(k) = 从顶点k到汇点的最短路径长度。

假设第 i 条弧为 <j, k>

则 对第 i 项活动言

事件发生时间的计算公式:

ve(源点) = 0;

ve(k) = Max{ve(j) + dut(<j, k>)}

vl(汇点) = ve(汇点);

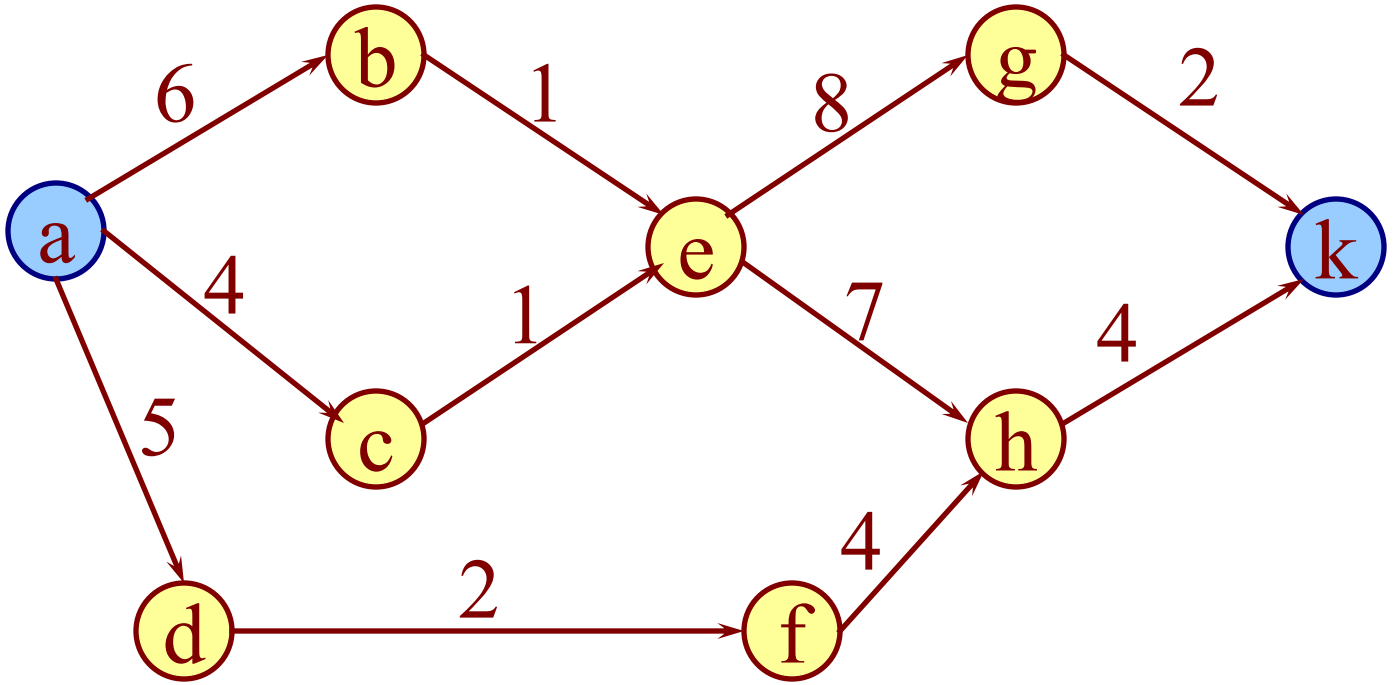
vl(j) = Min{vl(k) – dut(<j, k>)}

“活动(弧)”的 最早开始时间 $ee(i)$

$ee(i) = ve(j)$;

“活动(弧)”的 最迟开始时间 $el(i)$

$el(i) = vl(k) - dut(<j,k>)$;



	a	b	c	d	e	f	g	h	k
ve	0	6	4	5	7	7	15	14	18
vl	0	6	6	8	7	10	16	14	18

拓扑有序序列：a - d - f - c - b - e - h - g - k

	a	b	c	d	e	f	g	h	k
ve	0	6	4	5	7	7	15	14	18
vl	0	6	6	8	7	10	16	14	18

	ab	ac	ad	be	ce	df	eg	eh	fh	gk	hk
权	6	4	5	1	1	2	8	7	4	2	4
e	0	0	0	6	4	5	7	7	7	15	14
l	0	2	3	6	6	8	8	7	10	16	14
	✓			✓				✓			✓

算法的实现要点:

显然，求ve的顺序应该是按**拓扑有序**的次序；

而　求vl的顺序应该是按**拓扑逆序**的次序；

因为　拓扑逆序序列即为拓扑有序序列的**逆序列**，

因此　应该在拓扑排序的过程中，另设一个“**栈**”记下拓扑有序序列。

求关键路径的算法步骤为：

（1）输入e条弧<j,k>,建立AOE-网的存储结构；

（2）从源点v0出发，令ve[0] = 0，按拓扑有序求其余各顶点的最早发生时间ve[i](1≤i≤n-1)。如果得到的拓扑有序序列中顶点个数小于网中顶点数n，则说明网中存在环，不能求关键路径，算法终止；否则执行步骤（3）

（3）从汇点vn出发，令vl[n-1]=ve[n-1],按逆拓扑有序求其余各顶点的最迟发生时间vl[i](n-2≥i≥2)；

（4）根据各顶点的ve和vl值，求每条弧s的最早开始时间e(s)和最迟开始时间l（s）。若某条弧满足条件e(s)=l(s),则为关键活动。

```
int topologicalOrder(ALGraph Stack &T)
{ /* 有向网G采用邻接表存储结构，求各顶点事件的最早发生时间
  ve(全局变量)*/
  /* T为拓扑序列顶点栈，S为零入度顶点栈。*/
  /* 若G无回路，则用栈T返回G的一个拓扑序列，且函数值为OK，
  否则为ERROR。*/
  FindInDegree(G, indegree);       /* 对各顶点求入度
  indegree[0..vernum-1]*/
  InitStack(S);                    /*建零入度顶点栈S*/
  count = 0;  ve[0..G.vexnum-1] = 0;     /*初始化ve[ ]*/
  for (i=0; i<G.vexnum; i++) /*将初始时入度为0的顶点入栈
  {if (indegree[i]==0)  push(S,i);
 if (count<G. vexnum)  return 0; else return 1;
} /* TopologicalOrder*/
```



```
while (! StackEmpty(S)) {
  Pop (S, j); Push (T, j); ++ count;
 /* j号顶点入T栈并计数*/
  for (p = G.adjlist[j].firstedge; p; p=p->next)
   { k = p->adjvex;
    /*对j号顶点的每个邻接点的入度减1*/
    if (- - indegree[k] = = 0) Push(S, k);
    /*若入度减为0, 则入栈*/
    if (ve[j]+* (p->info)>ve[k])
       ve[k] = ve[j]+*(p->info);   // 取所有前驱约束中的最大值
   }
```

```
int Criticalpath(ALGraph G)
{ /* G为有向网，输出G的各项关键活动。*/
  InitStack(T);       /*建立用于产生拓扑逆序的栈T*/
  if (! TopologicalOrder (G,T) )   return 0;   /*该有向网有回路返回0*/
  vl[0..G.vexnum-1] = ve [G.vexnum-1];
    /* 初始化顶点事件的最迟发生时间*/
    /*逆拓扑有序过程
for ( j=0; j<G. vexnum; + +j)              /*求ee、 el和关键活动*/
     for (p=G.adjlist [j].firstedge; p; p = p->next)
      { k = p->adjvex;   dut= * (p->indo);
        ee = ve [j]；  el = vl [k] - dut;
        tag = (ee= =el) ?  '*' :' '   ; // 最早与最迟开始时间相同即为关键活动
        printf ( j,k,dut,ee,el,tag );              /*输出关键活动*/
      }
```



```
while (! StackEmpty (T) )
    /*按拓扑逆序求各顶点的vl值*/
for (Pop(T,j), p=G. adjlist[j].firstedge; p; p=p->next)
    { k=p->adjvex;  dut = * (p->info); // dut 是从 j 到 k 的活动工期
     if ( vl [k]-dut < vl [j] )  vl [j] = vl [k]  -  dut;
    }
```

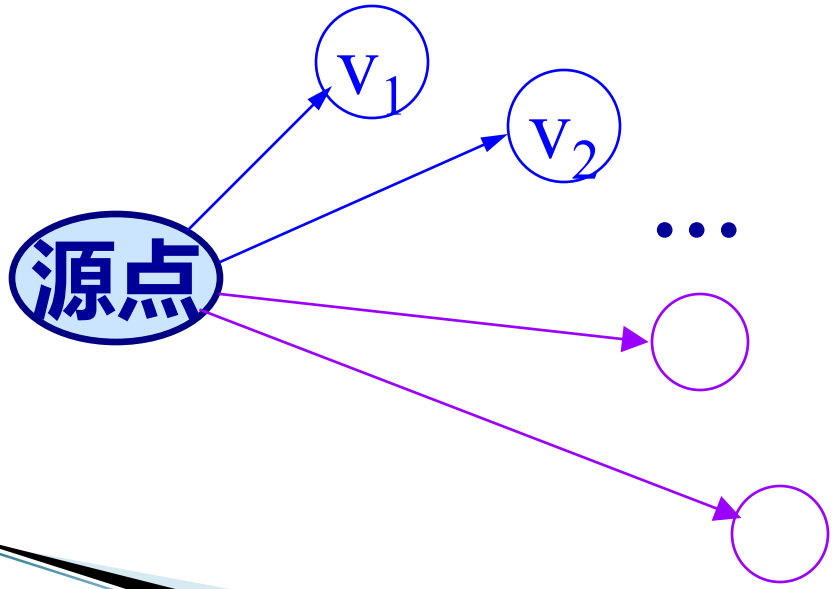
7.8 两点之间的最短路径问题

- 求从某个源点到其余各点的最短路径
- 每一对顶点之间的最短路径



求从源点到其余各点的最短路径的算法的基本思想:

依最短路径的长度递增的次序求得各条路径



其中，**从源点到顶点v的最短路径**是所有最短路径中长度最短者。

- 路径长度最短的**最短路径**的特点：

  在这条路径上，必定只含一条弧，并且这条弧的权值最小。

- 下一条路径长度次短的**最短路径**的特点：

  它只可能有两种情况：或者是直接从源点到该点(只含一条弧)； 或者是从源点经过顶点v_1，再到达该顶点(由两条弧组成)。

- 再下一条路径长度次短的**最短路径**的特点:

它可能有三种情况：或者是直接从源点到该点(只含一条弧)； 或者是从源点经过顶点v_1，再到达该顶点(由两条弧组成)；或者是从源点经过顶点v_2，再到达该顶点。

- 其余最短路径的特点：

它或者是直接从源点到该点(只含一条弧)； 或者是从源点经过已求得最短路径的顶点，再到达该顶点。

Dijkstra算法的正确性可以用反证法加以证明。

假设下一条最短路径的终点为x，那么，该路径必然或者是弧（v0，x），或者是中间只经过集合S中的顶点而到达顶点x的路径。因为假若此路径上除x之外有一个或一个以上的顶点不在集合S中，那么必然存在另外的终点不在S中而路径长度比此路径还短的路径，这与我们按路径长度递增的顺序产生最短路径的前提相矛盾，所以此假设不成立。

求最短路径的迪杰斯特拉算法：

设置辅助数组Dist，其中每个分量Dist[k]表示 当前所求得的从源点到其余各顶点 k 的最短路径。

一般情况下，

Dist[k] = <源点到顶点 k 的弧上的权值>

或者 = <源点到其它顶点的路径长度>

+ <其它顶点到顶点 k 的弧上的权值>。

1）在所有从源点出发的弧中选取一条权值最小的弧，即为第一条最短路径。

$$Dist[k]=\begin{cases} G.arcs[v0][k] & \text{V0和k之间存在弧} \\ INFINITY & \text{V0和k之间不存在弧} \end{cases}$$

其中的最小值即为最短路径的长度。

2）修改其它各顶点的$Dist[k]$值。

假设求得最短路径的顶点为u，

若 *Dist[u]+G.arcs[u][k]<Dist[k]*

则将 $Dist[k]$ 改为 $Dist[u]+G.arcs[u][k]$。

```
void ShortestPath_1(Mgraph G,int v0,PathMatrix *p, ShortPathTable
      *D)
{ /*用Dijkstra算法求有向网G的v0顶点到其余顶点v的最短路径P[v]
      及其路径长度D[v]*/
 /*若P[v][w]为TRUE，则w是从v0到 v当前求得最短路径上的顶点
 /*final[v] 为TRUE当且仅当v ∈ S, 即已经求得从v0到v的最短路径
 /*常量INFINITY为边上权值可能的最大值
 for (v=0;v<G.vexnum;++v)
    {fianl[v]=FALSE; D[v]=G.edges[v0][v];
     for (w=0; w<G.vexnum; ++w)  P[v][w]=FALSE;  /*设空路径*/
     if (D[v]<INFINITY) {P[v][v0]=TRUE; P[v][w]=TRUE;}
    }
D[v0]=0; final[v0]=TRUE;          /*初始化，v0顶点属于S集*/
```

```
/*开始主循环，每次求得v0到某个v 顶点的最短路径，并加v到集
  */
   for(i=1; i<G.vexnum; ++i)          /*其余G.vexnum-1个顶点*/
     {min=INFINITY;          /*min为当前所知离v0顶点的最近距离*/
      for (w=0;w<G.vexnum;++w)
        if (!final[w])                  /*w顶点在V - S中*/
         if (D[w]<min) {v=w; min=D[w];} // 挑选尚未确定的最近顶点
      final[v]=TRUE                          /*离v0顶点最近的v加入S集合*/
      for(w=0;w>G.vexnum;++w)     /*更新当前最短路径*/
    /*修改D[w]和P[w],w ∈ V-S*/
     if (!final[w]&&(min+G.edges[v][w]<D[w]))
           { D[w]=min+G.edges[v][w];
            P[w]=P[v]+P[w];P[w][v]=TRUE;
```

求每一对顶点之间的最短路径

弗洛伊德算法的基本思想是：

从 v_i 到 v_j 的所有可能存在的路径中，选出一条长度最短的路径。

若$<v_i,v_j>$存在，则存在路径$\{v_i,v_j\}$

// 路径中不含其它顶点

若$<v_i,v_1>$,$<v_1,v_j>$存在，则存在路径$\{v_i,v_1,v_j\}$

// 路径中所含顶点序号不大于1

若$\{v_i,\ldots,v_2\}$, $\{v_2,\ldots,v_j\}$存在，

则存在一条路径$\{v_i, \ldots, v_2, \ldots v_j\}$

// 路径中所含顶点序号不大于2

…

依次类推，则 vi 至 vj 的最短路径应是上述这些路径中，路径长度最小者。

现定义一个n阶方阵序列。

$D^{(-1)}$，$D^{(0)}$，$D^{(1)}$，…，$D^{(k)}$，$D^{(n-1)}$

其中

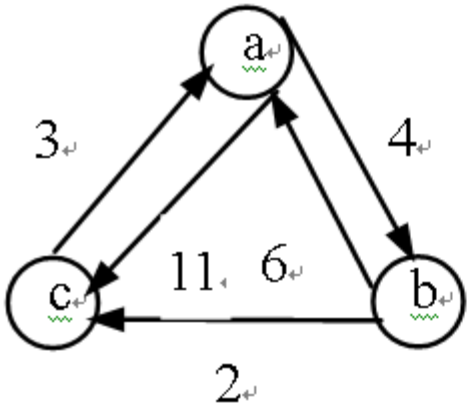
$D^{(-1)}[i][j]=edges[i][j]$

$D^{(k)}[i][j]=$

$$\text{Min}\{D^{(k-1)}[i][j], D^{(k-1)}[i][k]+D^{(k-1)}[k][j]\}$$

$$0 \leqq k \leqq n-1$$

从上述计算公式可见，$D^{(1)}[i][j]$是从vi到vj的中间顶点的序号不大于1的最短路径的长度；$D^{(k)}[i][j]$ 是从vi到vj的中间顶点的个数不大于k的最短路径的长度；$D^{(n-1)}[i][j]$ 就是从vi 到vj的最短路径的长度。



$$\begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{pmatrix}$$

$$D^{(-1)} = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{pmatrix} \quad D^{(0)} = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix} \quad D^{(1)} = \begin{pmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix} \quad D^{(2)} = \begin{pmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$$

$$P^{(-1)} = \begin{pmatrix} & ab & ac \\ ba & & bc \\ ca & & \end{pmatrix} \quad P^{(0)} = \begin{pmatrix} & ab & ac \\ ba & & bc \\ ca & cab & \end{pmatrix} \quad P^{(1)} = \begin{pmatrix} & ab & abc \\ ba & & bc \\ ca & cab & \end{pmatrix} \quad P^{(2)} = \begin{pmatrix} & ab & abc \\ bca & & bc \\ ca & cab & \end{pmatrix}$$

弗洛伊德算法

第 125–128 页给出问题定义、递推式和矩阵算例。补充原地更新距离矩阵的三重循环。调用方令对角线为 0，不可达位置为 FLOYD_INF。输入至少有一个顶点，图中无负权回路，有限路径长度位于 (-FLOYD_INF, FLOYD_INF)。

```
#include <limits.h>

#define FLOYD_INF (LLONG_MAX / 4)

void floyd(int n, long long d[n][n])
{
    for (int k = 0; k < n; ++k) {  // 本轮允许使用中间顶点 k
        for (int i = 0; i < n; ++i) {
            if (d[i][k] == FLOYD_INF) {
                continue;
            }
            for (int j = 0; j < n; ++j) {
                if (d[k][j] == FLOYD_INF) {
                    continue;  // 两段都可达时才尝试拼接
                }
                long long via_k = d[i][k] + d[k][j];
                if (via_k < d[i][j]) {
                    d[i][j] = via_k;
                }
            }
        }
    }
}
```

1. 熟悉图的各种存储结构及其构造算法，了解实际问题的求解效率与采用何种存储结构和算法有密切联系。

2. 熟练掌握图的两种搜索路径的遍历：遍历的逻辑定义、深度优先搜索和广度优先搜索的算法。

在学习中应注意图的遍历算法与树的遍历算法之间的类似和差异。

3. 应用图的遍历算法求解各种简单路径问题。

4. 理解教科书中讨论的各种图的算法。